阳光学院

人工智能学院

项目开发实训

期末文档

项目开发实训课程本课程是软件工程专业重要的集中性实践课程，在课程教学中，学生从项目招投标开始，完成需求分析、系统设计、系统实现、系统测试、系统部署等软件开发的全流程项目实践，是软件专业知识应用的综合实践。

班级：2024 级计算机科学与技术（专升本）5 班

课题名：在线旅游预订平台的开发与推广系统

项目团队名称：壮志凌云（黄鹏伟，谭敏静，吴佳程，程卓宇，许声鑫）

日期：2025 年 12 月

目　录

第 1 章 项目的背景意义 3
1.1 项目的背景 3
1.2 项目的意义 3
第 2 章 项目的需求分析 3
2.1 商家管理 3
2.2 商品管理 4
第 3 章 项目的设计 7
第 4 章 项目的实现 21
第 5 章 项目的测试 42
附录 项目的招投标文件 ..... 错误！未定义书签。

第 1 章 项目的背景意义

1.1 项目的背景

1.市场规模：2025 年预计突破 1.5 万亿元，用户规模达 5.5 亿
2.竞争格局：携程、同程、飞猪、美团四强主导，抖音、小红书等社交平台加速渗透
3.核心技术：AI 推荐准确率 75%，AR/VR 虚拟景区覆盖率 30%，微服务架构占比 80%
4.营销渠道：短视频贡献 40%新用户，KOL 预算 35%
5.最新政策：2025 年实施《在线旅游经营服务管理新规》
6.主要挑战：技术系统稳定性(99.99%)、数据孤岛；运营隐私保护、供应链管理；市场同质化竞争、用户忠诚度下降

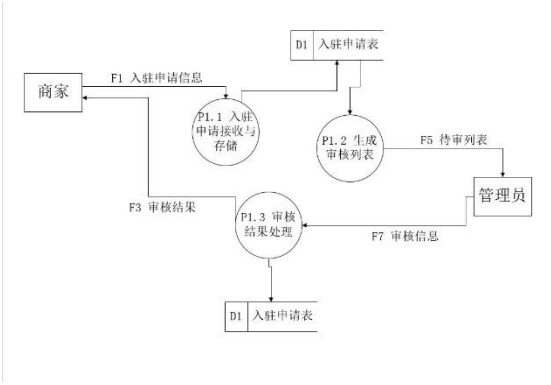
1.2 项目的意义

1.提升服务效率：通过自动化预订管理、智能行程规划等技术手段，降低人工操作成本，优化资源配置。
2.增强用户体验：提供个性化推荐、多语言客服等功能，满足游客多元化需求。
3.整合旅游资源：打通酒店、交通、景区等供应链，实现一站式服务，提升资源利用率。
4.拓展市场份额：通过数据分析精准定位市场，制定差异化营销策略，扩大品牌影响力

第 2 章 项目的需求分析

2.1 商家管理

负责商家的入驻申请、资质审核、信息维护及日常运营支持。
1） 入驻申请处理
2） 资质审核管理
3） 商家运营支持

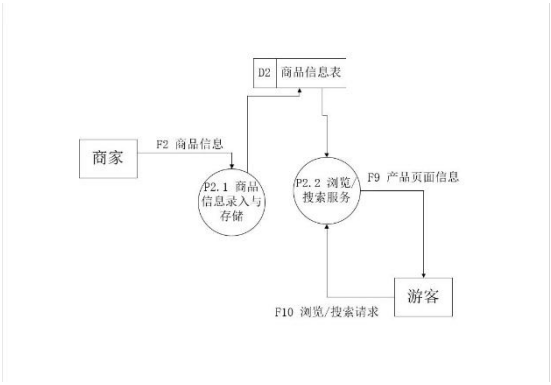


2.2 商品管理

负责旅游产品的信息管理、发布和展示。

1) 商品信息维护

2) 商品展示服务

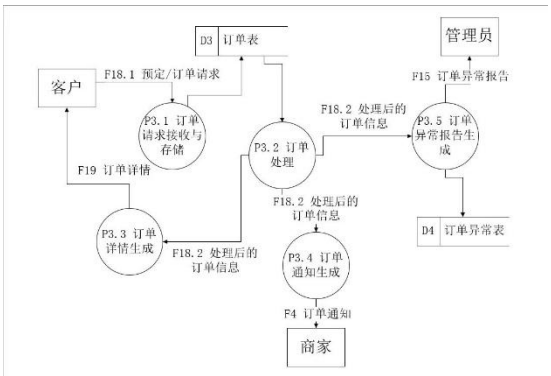


2.3 订单管理

负责订单的生成、状态跟踪和异常处理。

1) 订单生成与处理

2) 订单状态管理

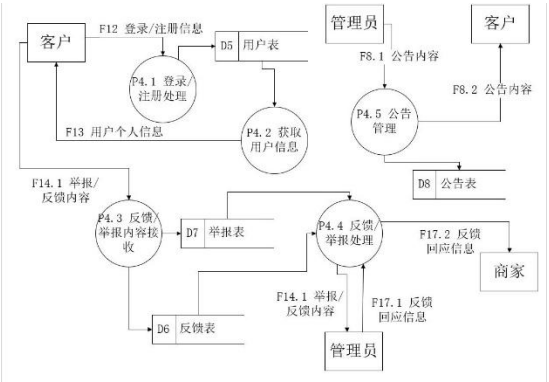


2.2 客户管理

负责客户账户管理、反馈处理和用户服务。

1) 用户账户管理

2) 客户服务支持



2.4 外部接口需求

2.4.1 用户界面

- a. 屏幕格式:
  - . 菜单：整体采用顶部导航栏与侧边栏相结合的布局，符合现代 Web 应用的操作习惯。
  - . 输入：系统的全部数据输入均采用标准 Web 表单控件，提供清晰的标签和输入提示，并进行前端实时校验。
  - . 输出：系统的查询结果主要以列表、卡片和详情页的形式在屏幕上进行输出。
- b. 报表打印格式： 商家后台的财务对账等模块，将提供导出为 Excel 或 PDF 的功能，格式将以简洁、清晰的表格为主。

2.4.2 硬件接口

本系统为纯软件系统，不直接与特定硬件交互。系统运行的硬件环境为通用服务器和个人计算机/移动设备，通过标准网络设备连接。

2.4.3 软件接口

支付接口： 后端需集成微信支付和支付宝的官方 SDK，调用其 API 完成支付流程。

短信服务接口： 后端需调用第三方短信服务的 API，用于发送手机验证码。

地图服务接口： 前端需调用地图服务商的 JS API，用于展示酒店地理位置

2.4.4 通讯接口

本系统前后端通讯采用基于 HTTP/S 的 TCP/IP 网络协议。

2.5 性能需求

a. 输入： 关键信息提供“常用旅客”选择功能，减少重复输入。
b. 处理： 核心交易流程要求在 2 秒内完成响应。商品搜索响应时间在 1 秒内。
c. 输出： 列表页面支持分页加载，避免一次性加载过多数据导致页面卡顿

2.6 设计约束

2.6.1 需求遵循的其它标准

- a. 报表格式： 参照主流 OTA 平台的订单和报表格式，由 UI 设计师进行设计。
- b. 数据命名：数据库表和字段命名遵循统一的、有意义的英文命名规范。

2.6.2 硬件的限制

系统开发完成后，可运行于任何安装了现代浏览器（Chrome, Firefox, Safari, Edge）的 PC 或移动设备上。

2.7 属性需求

2.7.1 可使用性

要求具有较强的容错处理能力，以保证软件的可使用性。在用户误操作的情况下，系统应给出明确、友好的提示信息。

2.7.2 安全性

- 用户密码必须经过哈希加盐后存储，防止明文泄露。
- 所有 API 接口需进行严格的认证与授权校验，防止未授权访问。
- 有效防范 OWASP Top 10 中常见的 Web 安全漏洞，如 SQL 注入、XSS 等。

2.7.3 可维护性

为了保证系统的可维护性，要求具有详细的系统设计文档资料，提供全部的系统源程序，并采用模块化的代码结构，降低耦合度。

2.7.4 可移植性

本系统后台基于 Java 开发，可移植到任何支持 JVM 的操作系统。前端基于 Web 标准，可在所有现代浏览器上运行。

第 3 章 项目的设计

3.1 总体设计

本项目所开发的在线旅游预订平台是一个多角色、多端协同的综合性在线旅游预订平台。它旨在通过整合旅游产品资源，并利用智能推荐和社区分享功能，解决当前旅游工具分散、信息筛选困难的痛点。

3.2 软件描述

系统将涵盖客户应用、商家应用和管理员应用三大核心部分，具体包括：登录注册子系统、主页与搜索子系统、心动模式子系统、订单管理子系统、个人信息子系统、社区功能子系统、商家管理子系统和平台管理子系统。

3.2.1 各子系统的主要业务

1）登录注册子系统： 包括手机号/密码注册、手机号登录、密码找回等功能。

2）主页与搜索子系统： 包括首页内容推荐、全局搜索、车票预订等。

3）心动模式子系统： 核心创新模块。通过算法向用户推送个性化的目的地或旅行团卡片，用户可通过滑动进行浏览和预订。

4）个人信息与订单子系统：

1. 个人信息： 包括用户个人资料管理、收藏夹、足迹浏览。
2. 订单管理： 根据“逻辑分离”原则，路线、酒店、机票等产品将被独立下单。本子系统负责处理订单的创建、支付、状态跟踪和详情查看。
3. 旅客管理： 提供“常旅客”信息的统一管理功能，用于在预订机票、火车票时快速填充信息。

5)社区子系统： 包括用户浏览、搜索、发布、评论、点赞游记帖子的功能。

6) 行程子系统： 包括查看当日行程信息、查看行程日历等功能。

7) 商家管理子系统： 包括商家入驻申请、提交资质、商品信息的录入与管理、订单状态管理、审核结果查询等。

8) 平台管理子系统： 包括商家入驻申请的审核、订单与异常管理、平台反馈与公告管理等。

3.2.2 主要输入/输出

a、动态输入数据:

1) 用户登录凭据。
2) 用户的搜索关键词、筛选条件。
3) 商家的入驻信息和发布的商品信息。
4) 用户的预订信息。
5) 用户在社区发布的帖子和评论内容。

b、动态输出数据：

1) 旅游产品列表、景点信息。
2) “心动模式”推送的个性化内容。
3) 订单详情、订单状态、支付结果。
4) 社区帖子列表、评论列表。
5) 商家审核结果、商品管理状态。

3.2.3 运行环境

1) 硬件环境

服务器:腾讯云 ECS。

客户端: 支持现代浏览器的 PC 机，以及主流的 iOS 和 Android 移动设备。

2) 软件环境

操作系统: Linux

数据库管理系统: Windows, macOS, iOS, Android

web 服务器：MySQL

编程语言：Nginx

后端编程语言： Java

后端运行环境： Spring Boot

前端编程语言： Vue 3 + TypeScript

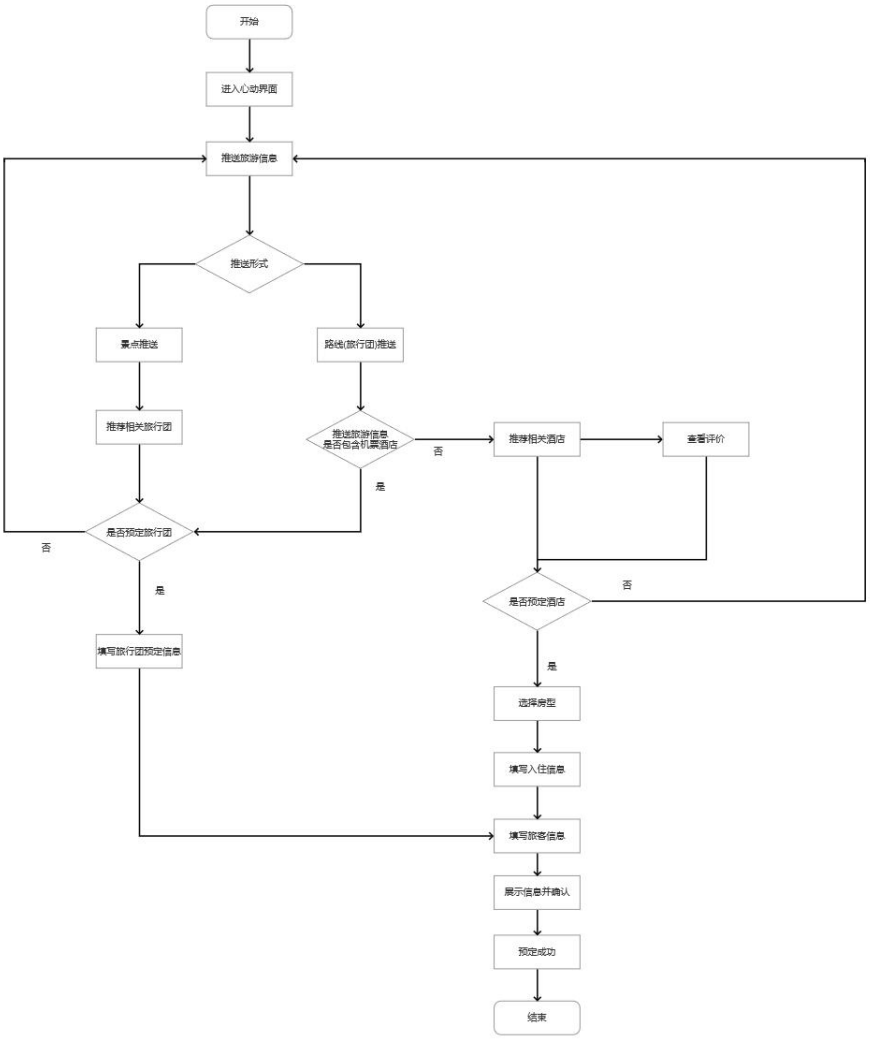
前端运行环境：

框架: unibest

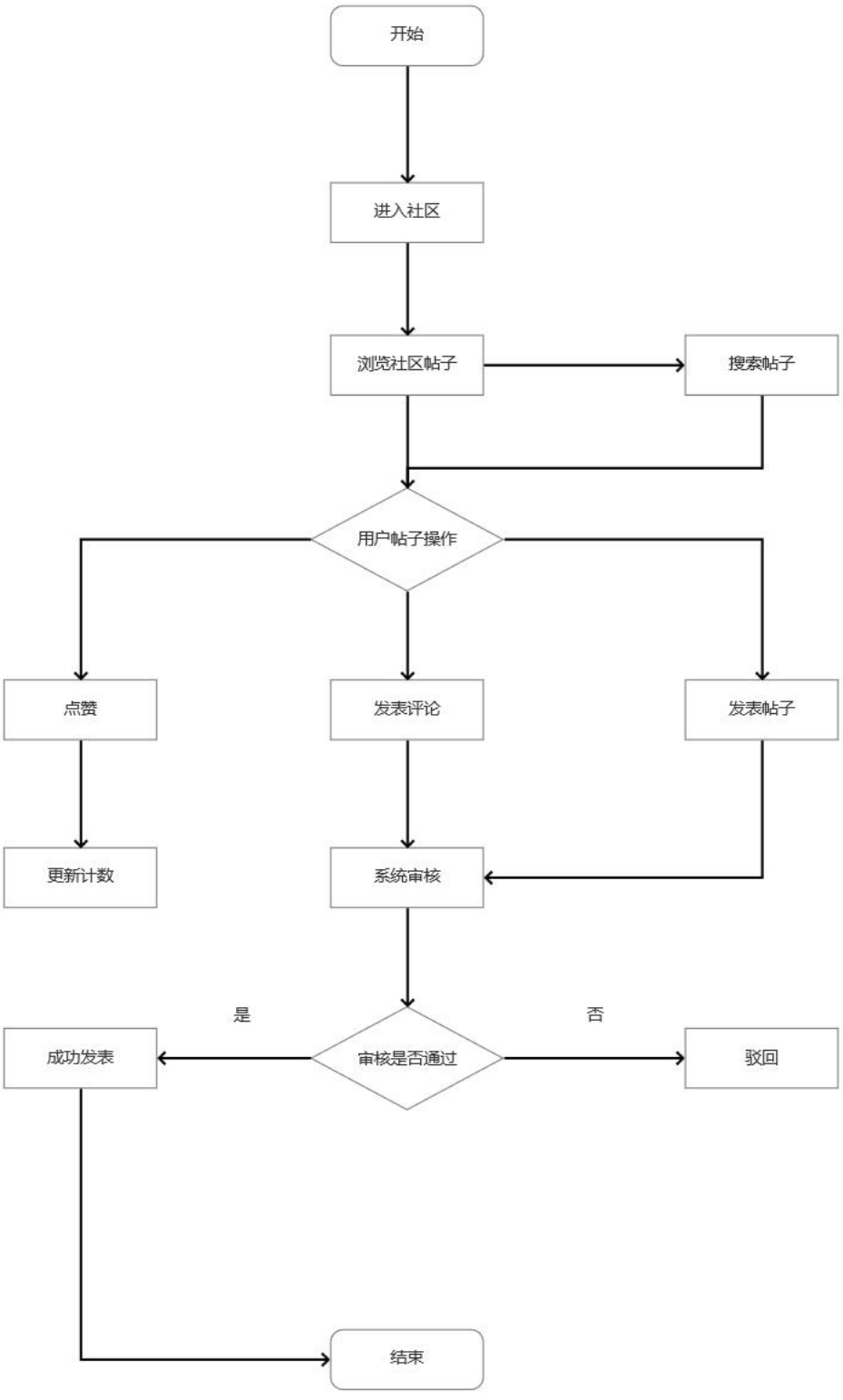
UI库: Wot Design Uni + UnoCSS

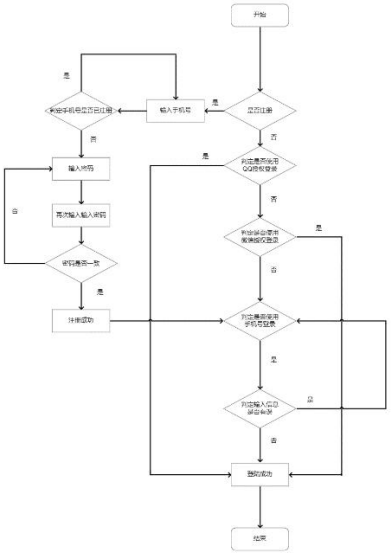
构建工具: Vite + pnpm

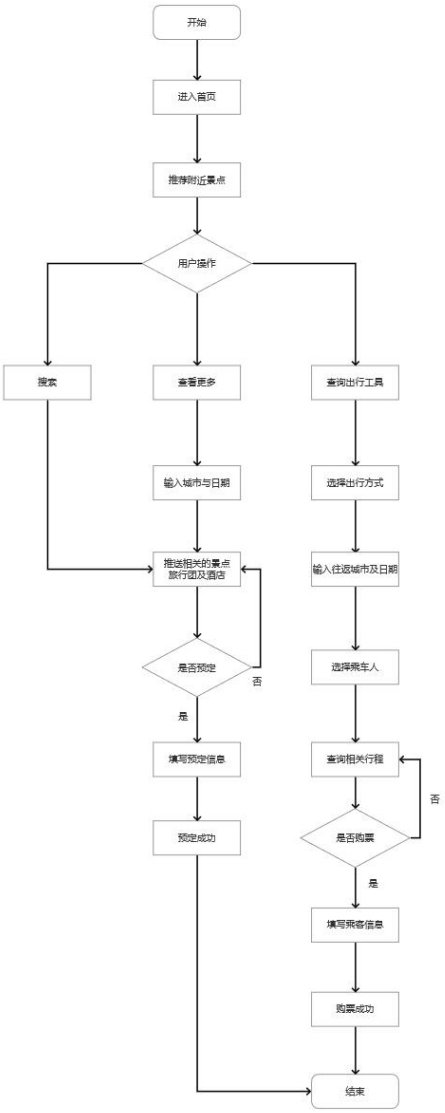
3.3 处理流程


开始
进入心动界面
推送旅游信息
推送形式
景点推送
路线(旅行团)推送
推荐相关旅行团
推送旅游信息
是否包含机票酒店
否
是
推荐相关酒店
查看评价
是否预定旅行团
否
是
是否预定酒店
否
是
填写旅行团预定信息
选择房型
填写入住信息
填写旅客信息
展示信息并确认
预定成功
结束

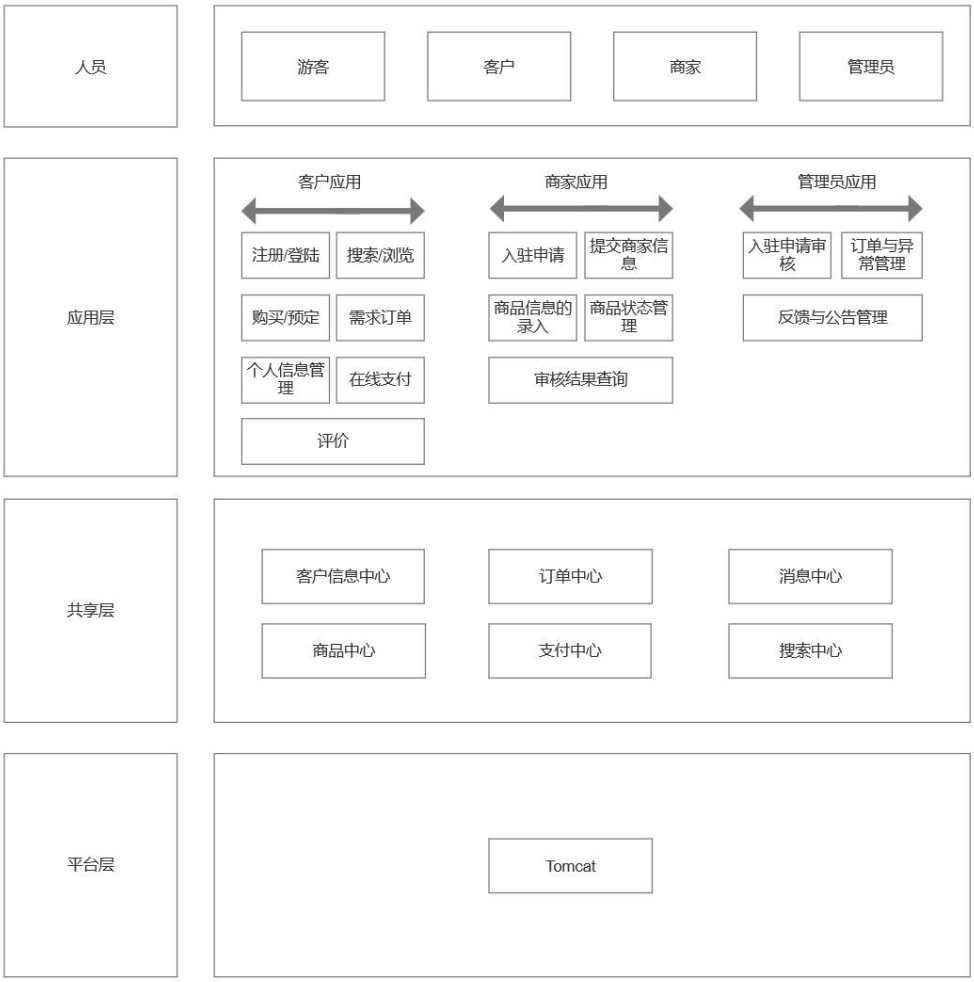

开始
进入行程页
查看当日行程信息
查看行程日历
结束


开始
进入社区
浏览社区帖子
搜索帖子
用户帖子操作
点赞
发表评论
发表帖子
更新计数
系统审核
是
否
成功发表
审核是否通过
驳回
结束


开始
是否注册
是
输入手机号
是
该手机号是否已经
是
否
输入密码
否
再次输入输入密码
密码是否一致
是
注册成功
否
是否是第三方
QQ或者微博
是
否
是否是会员也
有实体店位置
是
否
是否是第三方
手机号登录
是
是否输入过位
是
置信息
否
登陆成功
结束


开始
进入首页
推荐附近景点
用户操作
搜索
查看更多
查询出行工具
输入城市与日期
选择出行方式
推送相关的景点
旅行团及酒店
输入往返城市及日期
是否预定
否
选择乘车人
是
填写预定信息
查询相关行程
预定成功
否
是否购票
是
填写乘客信息
购票成功
结束

3.4 软件结构



3.5 模块外部设计

3.5.1 功能



1） 登录注册功能

维护用户的账户体系，提供手机、微信、QQ等多种登录注册方式，并处理密码找回。

2) 个人信息功能

维护用户的个人资料、收藏、足迹、订单列表和常旅客信息。

3) 主页功能

作为应用的核心入口，提供内容推荐、全局搜索和车票预订的快捷入口。

4) 社区功能

负责用户生成内容，包括浏览、搜索、发布、评论和点赞帖子。

5) 行程功能

负责向用户展示日历维度的行程安排和当日行程详情。

6) 心动模式功能

负责算法推荐和“瀑布流”式的内容推送与预订。

7) 商家管理功能

供商家使用的独立后台，核心功能包括入驻申请、商品管理、订单管理。

8) 系统管理功能

供平台管理员使用，核心功能包括商家资质审核、订单异常处理、平台公告发布。

3.5.2 性能

本系统采用 unibest 框架，其内置的 UnoCSS 引擎能确保CSS包体积最小化。UI组件库 Wot-UI 支持按需加载，进一步优化了前端性能和加载速度。系统界面设计遵循设计规范，确保操作响应及时，在主流移动设备上运行流畅。

3.5.3 代码设计

代码设计遵循开发规范文档中的约定，基于 ESLint 和 Prettier 强制执行统一的代码风格。unibest 框架提供了约定式路由和组件/API自动导入功能，极大地简化了代码的组织结构。数据库代码设计遵循4.2中的关系模式。

3.6 程序模块设计

本系统的模块设计是采用结构化设计的思想，自顶向下逐层分解；每个模块具有一定的功能和独立性。 模块设计的方法和原则，是在逻辑设计阶段数据流程图的基础上，本着模块要彼此独立、功能要明确、能单独进行修改和维护，而又不影响其他模块的设计原则；将一个庞大而又复杂的系统转化分解为许多简单模块。

3.7 接口设计

3.7.1 外部接口

本系统的开发模式采用前后端分离的方式，即前端 uni-app 应用通过 RESTful API (HTTP/S) 与后端 Spring Boot 服务器进行数据交换。同时，后端服务器将聚合以下第三方外部接口：

支付接口：微信支付官方SDK,支付宝官方SDK

短信服务接口：第三方短信服务商API，用于发送手机验证码。

地图服务接口：腾讯地图API，用于展示酒店、景点的地理位置。

旅游数据API：大型OTA开放平台API，用于获取酒店、机票、景点、火车票的实时信息。

3.7.2 内部接口

a.输入项

1） 商家管理

输入项名称	输入方式	数量和频度	输入介质	输入数据来源
商家入驻信息	Web 表单	中频，每日数十条	浏览器	商家用户

2）商品管理

输入项名称	输入方式	数量和频度	输入介质	输入数据来源
商品信息	Web 表单/API	高频，每日数百条	手动	商家用户

3）订单管理

输入项名称	输入方式	数量和频度	输入介质	输入数据来源
订单请求	Web 表单/API	高频，每日数千条	浏览器/移动端	客户用户

4）客户管理

输入项名称	输入方式	数量和频度	输入介质	输入数据来源
用户注册信息	Web 表单	中频，每日数百条	浏览器	客户用户

5）系统管理

输入项名称	输入方式	数量和频度	输入介质	输入数据来源
管理员操作指令	Web 表单	低频，每日数条	浏览器	管理员

b.输出项

1）商家管理

输出项名称	输出形式	数量和频度	输出介质	输出格式
审核结果通知	消息/邮件	中频	浏览器/邮件	JSON/HTML

2）商品管理

输出项名称	输出形式	数量和频度	输出介质	输出格式
商品列表	API 响应	高频	浏览器/移动端	JSON

3）订单管理

输出项名称	输出形式	数量和频度	输出介质	输出格式
订单状态	API 响应	高频	浏览器/移动端	JSON

4）客户管理

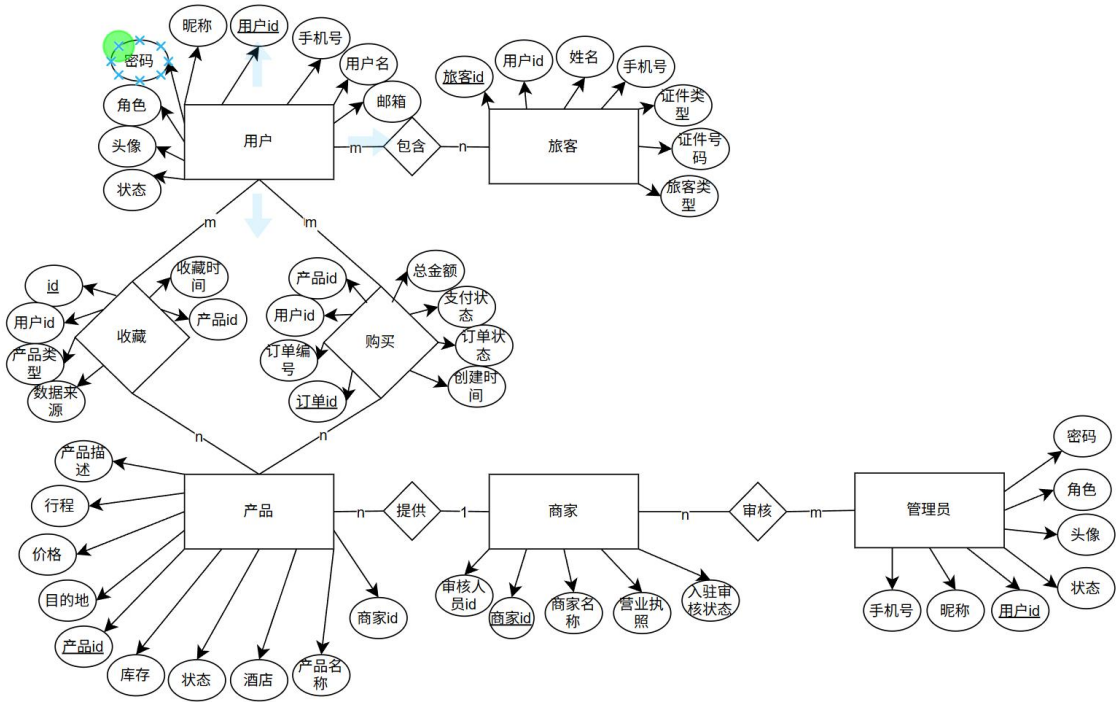
输出项名称	输出形式	数量和频度	输出介质	输出格式
用户信息	API 响应	中频	浏览器	JSON

5）系统管理

输出项名称	输出形式	数量和频度	输出介质	输出格式
操作日志	文件/数据库	低频	服务器	CSV/JSON

3.8、数据结构设计

3.8.1、E-R 图



3.8.2、关系模式

用户表（用户 ID，用户名，昵称，密码，手机号，邮箱，角色，头像，状态）
商家表（商家 ID，商家名称，营业执照，入驻审核状态，审核人员 ID）
产品表（产品 ID，产品名称，目的地，价格，行程，商家 ID，产品描述，库存，状态，酒店）
订单表（订单 ID，订单编号，用户 ID，产品 ID，总金额，支付状态，订单状态，创建时间）
订单项表（ID，订单编号，产品 ID，数量）
旅客表（ID，关联用户 ID，姓名，电话，证件类型，证件号码，旅客类型）
用户收藏表（收藏 ID，产品 ID，用户 ID，产品类型，项目来源，创建时间）

3.8.3、物理结构设计

1、用户表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
user_id	int			☑	☐	🔑 1	用户id
user_name	char	100		☑	☐		用户名
nickname	varchar	255		☐	☐		昵称
password	varchar	255		☐	☐		密码
tel	varchar	11		☐	☐		电话
email	varchar	255		☐	☐		邮箱
role	enum			☑	☐		角色身份
avatar_url	varchar	255		☑	☐		用户头像
status	tinyint	1		☑	☐		用户状态

2、商家表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
user_id	int			☑	☐	🔑1	用户id（商家）
company_name	varchar	100		☑	☐		商家名称
business_license_url	varchar	255		☑	☐		营业执照
verification_status	tinyint			☑	☐		入驻审核状态
approved_by	int			☐	☐		审核人员（管理员）id

3、产品表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
product_id	int			☑	☐	🔑1	产品id
product_name	varchar	255		☑	☐		产品名称
destination_name	varchar	255		☑	☐		目的地
price	decimal	10	2	☑	☐		价格
ltinerary	varchar	255		☑	☐		行程
user_id	int			☑	☐		用户id（商家）
description	varchar	255		☐	☐		产品描述
stock	int			☑	☐		库存
status	tinyint			☑	☐		状态
hotel	tinyint	1		☑	☐		酒店

4、订单表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
order_id	int			☑	☐	🔑1	订单id
order_sn	bigint			☑	☐		订单编号
user_id	int			☑	☐		用户id（购买者）
product_id	int			☑	☐		产品id
total_amount	decimal	10	2	☑	☐		总金额
payment_status	tinyint	1		☑	☐		支付状态
order_status	tinyint			☑	☐		订单状态
created_at	datetime			☑	☐		创建时间

5、订单项表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	int			☑	☐	🔑1	
order_id	int			☑	☐		订单编号
product_id	int			☑	☐		产品id
quantity	int			☑	☐		数量

6、旅客表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	int			☑	☐	🔑1	旅客id
user_id	int			☑	☐		用户id
full_name	varchar	100		☑	☐		旅客姓名
phone	varchar	11		☑	☐		电话
id_card_type	tinyint			☑	☐		证件类型
id_card_number	varchar	50		☑	☐		证件号
traveler_type	tinyint			☑	☐		旅客类型

7、收藏表（产品收藏）

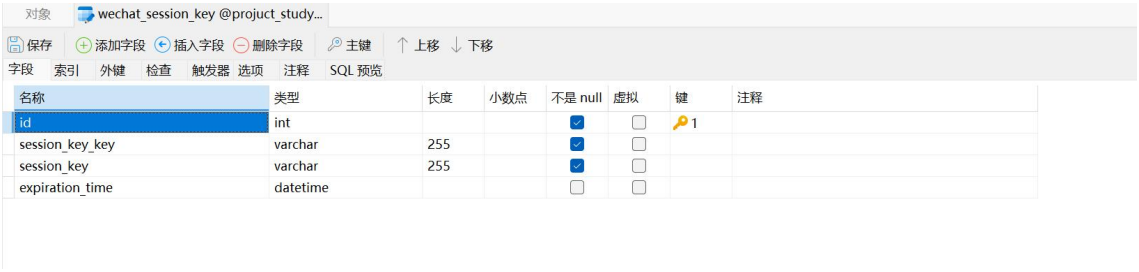
字段 索引 外键 检查 触发器 选项 注释 SQL 预览

名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	int			☑	☐	🔑1	
user_id	int			☑	☐		用户id（一般客户）
item_type	varchar	20		☑	☐		收藏的商品类型：'product', 'attraction', 'hotel'
item_id	int			☑	☐		商品id: 如果是自有产品则为products.id，第三方则为
item_provider	varchar	50		☑	☐		数据来源方 ('self', 'amap', 'ctrip')
created_at	datetime			☑	☐		该收藏创建时间

8、微信授权登录表

名称	类型	长度	小数点	不是 null	虚拟	键	注释
id	int			☑	☐	🔑 1	
user_id	int			☑	☐		
open_id	varchar	255		☑	☐		
avatar_url	varchar	255		☐	☐		
nick_name	varchar	255		☐	☐		

9、微信会话密钥表



3.9 运行设计

3.9.1 运行模块的组合

本系统的运行模块之间的组合由系统的总控菜单或各子系统的子菜单确定与控制，其具体功能参见 2.3 的本系统的软件结构。本系统的各子系统可独立运行，也可由系统的总控菜单调度统一运行。

3.9.2 运行控制

本系统的运行控制包括两个方面：

1、 系统的处理流程： 由 uni-app 的前端路由和 Wot-UI 提供的导航组件控制。系统提供清晰的、符合现代移动应用操作习惯的交互界面。

2、 系统的控制权限： 统一由后端 Spring Security 结合 JWT 来进行控制与管理。后端 API 将基于用户的角色进行严格的权限校验，实现 RBAC。

3.10 系统出错处理设计

3.10.1 出错信息

本系统由专门的出错处理程序完成出错信息的提示和系统的恢复处理。

后端:由 Spring Boot 的全局异常处理器捕获业务异常，向前端返回统一格式的 JSON 错误信息。

前端:在 uni-app 的 API 请求封装层统一拦截后端的错误响应，并调用 Wot-UI 的 wd-toast 或 wd-message-box 向用户显示清晰、友好的汉字提示信息。

3.10.2 补救措施

a、数据备份：本系统的数据备份和转贮功能由系统数据库管理员定期进行，一旦数据丢失，可随时利用所备份的数据进行备份的恢复。

b、恢复及再启动：一旦系统运行出错，前端应用将引导用户返回安全页面，后端服务基于 Spring Boot 的内嵌 Tomcat 运行，具备高可用性，可由运维脚本或容器编排工具实现故障自动重启。

第 4 章 项目的实现

4.1、手机短信验证码登录注册

```java
package com.febuki.tool.server.service.impl;

import ...

@Slf4j
@Service("smsAuthServer")
public class SmsAuthServerImpl extends ServiceImpl<SMSAuthMapper, SMSAuth> implements SmsAuthServer {

    private final JwtTokenUtil jwtTokenUtil;  2 个用法
    private final UserServer userService;  3 个用法
    private final SmsCodeCacheServer smsCodeCacheServer;  4 个用法

    public SmsAuthServerImpl(@Autowired JwtTokenUtil jwtTokenUtil, @Autowired UserServer userService, @Autowired SmsCodeCacheServer smsCodeCacheServer) {
        this.jwtTokenUtil = jwtTokenUtil;
        this.userService = userService;
        this.smsCodeCacheServer = smsCodeCacheServer;
    }

    private static final String SMS_CODE_PREFIX = "SMS_CODE:";  0 个用法
    private static final long SMS_CODE_EXPIRE = 300; // 5分钟  0 个用法

    /**
     * 生成验证码
     */
    @Override  1 个用法
    public String generateCode() { return smsCodeCacheServer.generateCode(); }

    /**
     * 发送短信验证码
     */
    @Override  1 个用法
    public String sendSmsCode(String phone) {
        log.info("开始发送短信验证码 - 手机号: {}", phone);

        // 验证手机号格式
        if (!isValidPhone(phone)) {
            log.error("手机号格式错误: {}", phone);
            throw new IllegalArgumentException("手机号格式错误");
        }


        try {
            // 生成验证码
            String code = generateCode();
            log.info("生成短信验证码成功 - 手机号: {}, 验证码: {}", phone, code);

            // 模拟发送短信（实际项目中替换为真实的短信服务）
            boolean sendResult = mockSendSms(phone, code);
            if (!sendResult) {
                log.error("短信发送失败 - 手机号: {}", phone);
                return null;
            }

            // 存储验证码到缓存
            smsCodeCacheServer.saveSmsCode(phone, code);
            log.info("短信验证码发送成功 - 手机号: {}", phone);

```

```
            return code;
        } catch (Exception e) {
            log.error("发送短信验证码异常 - 手机号: {}", phone, e);
            return null;
        }
    }

    /**
     * 模拟发送短信
     */
    private boolean mockSendSms(String phone, String code) {  1 个用法
        try {
            // 模拟网络延迟
            Thread.sleep( millis: 100);

            // 这里替换为实际的短信服务商API调用
            // 例如：阿里云短信、腾讯云短信等
            log.info("模拟发送短信 - 手机号: {}, 验证码: {}", phone, code);
            System.out.println("【模拟短信】发送验证码到 " + phone + ": " + code);

            return true;
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log.error("模拟发送短信被中断", e);
            return false;
        } catch (Exception e) {
            log.error("模拟发送短信异常", e);
            return false;
        }
    }

    /**
     * 验证短信验证码
     */
    @Override  1 个用法
    public boolean verifySmsCode(String phone, String code) {
        log.info("开始验证短信验证码 - 手机号: {}, 验证码: {}", phone, code);

        if (!isValidPhone(phone)) {
            log.error("手机号格式错误: {}", phone);
            throw new IllegalArgumentException("手机号格式错误");
        }

        if (!StringUtils.hasText(code)) {
            log.error("验证码为空 - 手机号: {}", phone);
            throw new IllegalArgumentException("验证码不能为空");
        }

        boolean isValid = smsCodeCacheServer.verifySmsCode(phone, code);
        if (!isValid) {
            throw new IllegalArgumentException("验证码错误或已过期");
        }

        return true;
    }

    /**
     * 短信登录
     */
    @Override  1 个用法
    public Map<String, Object> loginBySms(String phone, String code) {
        Map<String, Object> result = new HashMap<>();

        log.info("开始短信登录 - 手机号: {}", phone);

        try {
            // 验证手机号格式
            if (!isValidPhone(phone)) {
                result.put("success", false);
                result.put("message", "手机号格式不正确");
                return result;
            }

            // 验证验证码
            if (!verifySmsCode(phone, code)) {
                result.put("success", false);
                result.put("message", "验证码错误或已过期");
                return result;
            }

            // 查询或创建用户
            User user = userService.findByPhone(phone);
            if (user == null) {
                log.info("用户不存在，自动注册 - 手机号: {}", phone);
                // 自动注册
                user = new User();
                user.setTel(phone);
                user.setUsername(generateUsername(phone));
                userService.saveUser(user);
                log.info("用户自动注册成功 - 手机号: {}, 用户ID: {}", phone, user.getId());
            }

            // 检查用户状态
            if (!user.isStatus()) {
                result.put("success", false);
                result.put("message", "账号已被禁用");
                return result;
            }

            // 生成短信登录token
            String token = jwtTokenUtil.generateSmsToken(user.getId(), phone);

            result.put("success", true);
            result.put("token", token);
            result.put("user", user);
            result.put("loginType", "sms");
```

```
            log.info("短信登录成功 - 手机号: {}, 用户ID: {}", phone, user.getId());

        } catch (IllegalArgumentException e) {
            log.warn("短信登录验证失败 - 手机号: {}, 原因: {}", phone, e.getMessage());
            result.put("success", false);
            result.put("message", e.getMessage());
        } catch (Exception e) {
            log.error("短信登录异常 - 手机号: {}", phone, e);
            result.put("success", false);
            result.put("message", "系统异常，请稍后重试");
        }

        return result;
    }

    /**
     * 生成用户名
     */
    private String generateUsername(String phone) { return "用户_" + phone.substring( beginIndex: phone.length() - 4); }

    /**
     * 验证手机号格式
     */
    private boolean isValidPhone(String phone) {  3 个用法
        if (!StringUtils.hasText(phone) || phone.length() != 11) {
            return false;
        }
        return phone.matches( regex: "^1[3-9]\\d{9}$");
    }

    // 以下方法不再需要，因为使用缓存替代了数据库操作
    @Override  0 个用法
    public SMSAuth findByKey(String key) { throw new UnsupportedOperationException("findByKey 该方法已废弃，请使用缓存版本"); }

    @Override  0 个用法
    public SMSAuth updateByKey(String key) { throw new UnsupportedOperationException("updateByKey 该方法已废弃，请使用缓存版本"); }

    @Override  0 个用法
    public void deleteByKey(String key) { throw new UnsupportedOperationException("deleteByKey 该方法已废弃，请使用缓存版本"); }
}
```

```java
package com.febuki.tool.server.service.impl;

import ...

@Slf4j
@Service("smsCodeCacheServer")
public class SmsCodeCacheServerImpl implements SmsCodeCacheServer {

    private final CacheManager smsCacheManager;  4 个用法
    private static final String SMS_CODE_CACHE_NAME = "smsCode";  3 个用法

    public SmsCodeCacheServerImpl(@Qualifier("smsCacheManager") CacheManager smsCacheManager) {
        this.smsCacheManager = smsCacheManager;
    }

    /**
     * 生成6位数字验证码
     */
    @Override  1 个用法
    public String generateCode() {
        Random random = new Random();
        return String.format("%06d", random.nextInt( bound: 999999));
    }

    /**
     * 存储短信验证码
     */
    @Override  1 个用法
    public void saveSmsCode(String phone, String code) {
        try {
            Cache cache = smsCacheManager.getCache(SMS_CODE_CACHE_NAME);
            if (cache != null) {
                cache.put(phone, code);
                log.info("短信验证码已缓存 - 手机号: {}, 验证码: {}", phone, code);
            } else {
                log.error("获取短信验证码缓存失败");
            }
        } catch (Exception e) {
            log.error("存储短信验证码到缓存失败", e);
            throw new RuntimeException("存储验证码失败", e);
        }
    }

    /**
     * 获取短信验证码
     */
    @Override  2 个用法
    public String getSmsCode(String phone) {
        Cache cache = smsCacheManager.getCache(SMS_CODE_CACHE_NAME);
        if (cache == null) {
            log.error("短信验证码缓存未初始化");
            return null;
        }

        Cache.ValueWrapper valueWrapper = cache.get(phone);
        return valueWrapper != null ? (String) valueWrapper.get() : null;
```

```
    }

    /**
     * 验证短信验证码
     */
    @Override  1 个用法
    public boolean verifySmsCode(String phone, String code) {
        String storedCode = getSmsCode(phone);
        if (storedCode == null) {
            log.warn("验证码不存在或已过期 - 手机号: {}", phone);
            return false;
        }

        boolean isValid = storedCode.equals(code);
        if (isValid) {
            // 验证成功后删除缓存
            deleteSmsCode(phone);
            log.info("短信验证码验证成功 - 手机号: {}", phone);
        } else {
            log.warn("短信验证码验证失败 - 手机号: {}, 输入: {}, 期望: {}", phone, code, storedCode);
        }

        return isValid;
    }

    /**
     * 删除短信验证码
     */
    @Override  1 个用法
    public void deleteSmsCode(String phone) {
        Cache cache = smsCacheManager.getCache(SMS_CODE_CACHE_NAME);
        if (cache != null) {
            cache.evict(phone);
            log.info("短信验证码已删除 - 手机号: {}", phone);
        }
    }

    /**
     * 检查手机号是否已发送验证码
     */
    @Override  0 个用法
    public boolean hasSmsCode(String phone) { return getSmsCode(phone) != null; }
}
```

```java
package com.febuki.tool.server.controller;

import ...

@Slf4j
@RestController
@RequestMapping("/api/auth/sms")
@Tag(name = "短信验证码服务", description = "短信验证码服务")
public class SmsAuthController {

    private final SmsAuthServer smsAuthService;  3 个用法

    public SmsAuthController(@Autowired SmsAuthServer smsAuthService) { this.smsAuthService = smsAuthService; }

    @PostMapping("/sendCode")
    @Operation(summary = "发送短信验证码", description = "发送短信验证码")
    public ResponseEntity<?> sendSmsCode(
            @Parameter(
                    description = "手机号",
                    example = "13800000000"
            )
            @RequestParam String phone
    ) {
        log.info("发送短信验证码: {}", phone);
        String code = smsAuthService.sendSmsCode(phone);
        if (code != null) {
            log.info("验证码发送成功: {}", phone);
            return ResponseEntity.status(HttpStatus.OK).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.OK.value()),
                            MapUtils.Pair.of("message", "验证码发送成功"),
                            MapUtils.Pair.of("smsCode", code)
                    )
            );
        } else {
            log.error("验证码发送失败: {}", phone);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.INTERNAL_SERVER_ERROR.value()),
                            MapUtils.Pair.of("message", "验证码发送失败")
                    )
            );
        }
    }

    @PostMapping("/login")
    @Operation(summary = "短信登录", description = "短信登录")
    public Map<String, Object> loginBySms(
            @Parameter(
                    description = "手机号",
                    example = "13800000000"
            )
            @RequestParam String phone,
            @Parameter(
                    description = "验证码",
                    example = "123456"
            )
            )
            @RequestParam String code
    ) {
        log.info("短信登录: {}, {}", phone, code);
        return smsAuthService.loginBySms(phone, code);
    }
}
```

4.2、火车服务

```java
package com.febuki.tool.server.service.impl;

import ...

@Service("trainManagementServer")
public class TrainServerImpl implements TrainServer {

    private final JuheConfig juheConfig;  3 个用法

    public TrainServerImpl(@Autowired JuheConfig juheConfig) { this.juheConfig = juheConfig; }

    @Override  1 个用法
    public TrainScheduleResponseDTO getTrainSchedule(TrainScheduleRequestDTO trainScheduleRequestDTO) throws IOException {
        HashMap<String, String> map = getStringHashMap(trainScheduleRequestDTO);

        URL url = new URL(String.format("%s?%s", juheConfig.getUrl() + "fapigw/train/query", params(map)));

        BufferedReader in = new BufferedReader(new InputStreamReader((url.openConnection().getInputStream())));
        String inputLine;
        StringBuffer response = new StringBuffer();
        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();
        System.out.println(response);

        // 测试数据
        // String response = "{\"reason\":\"success.\",\"result\":[{\"train_no\":\"G103\",\"departure_station\":\"北京南\",\"arrival_station\":\"上海虹桥\",\"departu

        // Json 转换
        TrainScheduleResponseDTO trainScheduleResponseDTO = convertJsonToDto(response.toString());
        System.out.println(convertDtoToJson(trainScheduleResponseDTO));

        return trainScheduleResponseDTO;
    }

    private HashMap<String, String> getStringHashMap(TrainScheduleRequestDTO trainScheduleRequestDTO) {  1 个用法
        HashMap<String, String> map = new HashMap<>();
        map.put("key", juheConfig.getTrainApikey());
        map.put("search_type", trainScheduleRequestDTO.getSearchType());
        map.put("departure_station", trainScheduleRequestDTO.getDepartureStation());
        map.put("arrival_station", trainScheduleRequestDTO.getArrivalStation());
        map.put("date", trainScheduleRequestDTO.getDate());
        map.put("filter", trainScheduleRequestDTO.getFilter());
        map.put("enable_booking", trainScheduleRequestDTO.getEnableBooking());
        map.put("departure_time_range", trainScheduleRequestDTO.getDepartureTimeRange());
        return map;
    }

    private String params(Map<String, String> map) {  1 个用法
        return map.entrySet().stream() Stream<Entry<...>>
                .map( Entry<String, String> entry -> {
                    try {
                        return entry.getKey() + "=" + URLEncoder.encode(entry.getValue(), StandardCharsets.UTF_8.toString());
                    } catch (Exception e) {
                        return entry.getKey() + "=" + entry.getValue();
```

```
                    return entry.getKey() + "=" + entry.getValue();
                }
            }) Stream<String>
            .collect(Collectors.joining( delimiter: "&"));
    }

    /**
     * 将DTO转换回JSON字符串（用于测试或序列化）
     *
     * @param jsonString
     * @return TrainScheduleResponseDTO
     */
    private TrainScheduleResponseDTO convertJsonToDto(String jsonString) {  1 个用法
        try {
            // 使用FastJSON解析JSON字符串
            return JSON.parseObject(jsonString, TrainScheduleResponseDTO.class);
        } catch (Exception e) {
            throw new RuntimeException("JSON转换失败: " + e.getMessage(), e);
        }
    }

    /**
     * 将DTO转换回JSON字符串（用于测试或序列化）
     *
     * @param jsonString
     * @return TrainScheduleResponseDTO
     */
    private TrainScheduleResponseDTO convertJsonToDtoWithFeatures(String jsonString) {  0 个用法
        try {
            return JSON.parseObject(
                    jsonString,
                    TrainScheduleResponseDTO.class,
                    Feature.AllowUnQuotedFieldNames,
                    Feature.IgnoreNotMatch,
                    Feature.AllowSingleQuotes
            );
        } catch (Exception e) {
            throw new RuntimeException("JSON转换失败: " + e.getMessage(), e);
        }
    }

    // 将DTO转换回JSON字符串（用于测试或序列化）
    private String convertDtoToJson(TrainScheduleResponseDTO dto) {  1 个用法
        return JSON.toJSONString(dto,  prettyFormat: true); // true表示格式化输出
    }
}
```

```
package com.febuki.tool.server.controller;

import ...

@Slf4j
@RestController
@RequestMapping("/api/train")
@Tag(name = "火车服务", description = "火车服务")
public class TrainController {

    private final TrainServer trainServer;  2 个用法

    public TrainController(@Autowired TrainServer trainServer) { this.trainServer = trainServer; }

    @GetMapping("/query")
    @Operation(summary = "获取列车站到站时刻表", description = "列车站到站时刻表")
    public ResponseEntity<?> query(
            @Parameter(
                    name = "参数", required = true
            )
            TrainScheduleRequestDTO trainScheduleRequestDTO
    ) throws IOException {
        TrainScheduleResponseDTO response = trainServer.getTrainSchedule(trainScheduleRequestDTO);

        return ResponseEntity.status(HttpStatus.OK).body(
                MapUtils.of(
                        MapUtils.Pair.of("code", HttpStatus.OK.value()),
                        MapUtils.Pair.of("message", "查询成功"),
                        MapUtils.Pair.of("data", response)
                )
        );
    }
}
```

4.3、景点服务

```java
package com.febuki.tool.server.service.impl;

import ...

@Slf4j
@Service("scenicServer")
public class ScenicServerImpl implements ScenicServer {

    private final JuheConfig juheConfig;  3 个用法

    public ScenicServerImpl(@Autowired JuheConfig juheConfig) { this.juheConfig = juheConfig; }

    /**
     * 获取景点信息
     *
     * @param scenicRequestDTO
     * @return ScenicResponseDTO
     */
    @Override  1 个用法
    public ScenicResponseDTO getScenicInfo(ScenicRequestDTO scenicRequestDTO) throws IOException {
        URL url = new URL(String.format("%s?%s", juheConfig.getUrl() + "fapigx/scenic/query", params(getStringHashMap(scenicRequestDTO))));
        BufferedReader in = new BufferedReader(new InputStreamReader((url.openConnection()).getInputStream()));
        String inputLine;
        StringBuffer response = new StringBuffer();
        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();
        System.out.println(response);

        // 解析JSON
        ScenicResponseDTO scenicResponseDTO = parseToDto(response.toString());
        System.out.println(toFormattedJson(scenicResponseDTO));

        return scenicResponseDTO;
    }

    private HashMap<String, String> getStringHashMap(ScenicRequestDTO scenicRequestDTO) {  1 个用法
        HashMap<String, String> map = new HashMap<>();
        map.put("key", juheConfig.getScenicApikey());
        map.put("word", scenicRequestDTO.getWord());
        map.put("num", scenicRequestDTO.getNum());
        map.put("page", scenicRequestDTO.getPage());
        map.put("province", scenicRequestDTO.getProvince());
        map.put("city", scenicRequestDTO.getCity());
        return map;
    }

    private String params(Map<String, String> map) {  1 个用法
        return map.entrySet().stream() Stream<Entry<...>>
                .map( Entry<String, String> entry -> {
                    try {
                        return entry.getKey() + "=" + URLEncoder.encode(entry.getValue(), StandardCharsets.UTF_8.toString());
                    } catch (Exception e) {
                        log.error("编码错误: {}={}", entry.getKey(), entry.getValue(), e);
                        return entry.getKey() + "=" + entry.getValue();
                    }
                }) Stream<String>
                .collect(Collectors.joining( delimiter: "&"));
    }

    private ScenicResponseDTO parseToDto(String jsonString) {  1 个用法
        try {
            return JSON.parseObject(jsonString, ScenicResponseDTO.class);
        } catch (Exception e) {
            System.err.println("JSON解析失败: " + e.getMessage());
            return null;
        }
    }

    private String toFormattedJson(ScenicResponseDTO dto) {  1 个用法
        return JSON.toJSONString(dto,
                SerializerFeature.PrettyFormat,
                SerializerFeature.WriteMapNullValue,
                SerializerFeature.WriteDateUseDateFormat);
    }

    private String cleanHtmlContent(String htmlContent) {  0 个用法
        if (htmlContent == null) return "";
        return htmlContent.replaceAll( regex: "<br>",  replacement: "\n")
                .replaceAll( regex: "<[^>]+>",  replacement: "")
                .trim();
    }
}
```

```java
package com.febuki.tool.server.controller;

import ...

@Slf4j
@RestController
@RequestMapping("/api/scenic")
@Tag(name = "景点服务", description = "景点服务")
public class ScenicController {

    private final ScenicServer scenicServer;

    public ScenicController(@Autowired ScenicServer scenicServer) { this.scenicServer = scenicServer; }

    @GetMapping("/query")
    @Operation(summary = "获取景点信息", description = "获取景点信息")
    public ResponseEntity<?> query(
            @Parameter(
                    name = "参数", required = true
            )
            ScenicRequestDTO scenicRequestDTO
    ) throws IOException {
        log.info("获取景点信息");
        ScenicResponseDTO response = scenicServer.getScenicInfo(scenicRequestDTO);
        return ResponseEntity.status(HttpStatus.OK).body(
                MapUtils.of(
                        MapUtils.Pair.of("code", HttpStatus.OK.value()),
                        MapUtils.Pair.of("message", "查询成功"),
                        MapUtils.Pair.of("data", response)
                )
        );
    }
}
```

4.4、微信登录

```java
package com.febuki.tool.server.service.impl;

import ...

@Slf4j
@Service("wechatAuthServer")
public class WeChatAuthServerImpl extends ServiceImpl<WeChatAuthMapper, WeChatAuth> implements WeChatAuthServer {

    private final WeChatConfig wechatConfig;  5 个用法
    private final JwtTokenUtil jwtTokenUtil;  5 个用法
    private final WeChatCacheServer weChatCacheServer;  7 个用法

    public WeChatAuthServerImpl(
            @Autowired WeChatConfig wechatConfig,
            @Autowired JwtTokenUtil jwtTokenUtil,
            @Autowired WeChatCacheServer weChatCacheServer
    ) {
        this.wechatConfig = wechatConfig;
        this.jwtTokenUtil = jwtTokenUtil;
        this.weChatCacheServer = weChatCacheServer;
    }

    @Override  0 个用法
    public String getSessionKey(String sessionKeyKey) {
        log.info("获取微信SessionKey - sessionKeyKey: {}", sessionKeyKey);
        String sessionKey = weChatCacheServer.getSessionKey(sessionKeyKey);
        if (sessionKey == null) {
            log.warn("微信SessionKey不存在或已过期 - sessionKeyKey: {}", sessionKeyKey);
            throw new RuntimeException("SessionKey不存在或已过期");
        }
        return sessionKey;
    }

    @Override  2 个用法
    public boolean validateToken(String token) {
        log.info("验证token: {}", token);
        return jwtTokenUtil.validateToken(token);
    }

    @Override  0 个用法
    public boolean isExpired(String sessionKeyKey) {
        boolean expired = !weChatCacheServer.hasValidSessionKey(sessionKeyKey);
        log.info("检查微信SessionKey是否过期 - OpenID: {}, 结果: {}", sessionKeyKey, expired);
        return expired;
    }

    @Override  1 个用法
    public void saveSessionKey(String sessionKeyKey, String sessionKey) {
        log.info("保存微信SessionKey - sessionKeyKey: {}", sessionKeyKey);
        // 微信session_key默认有效期为30天，这里设置缓存过期时间为29天，确保提前更新
        int expirationSeconds = 29 * 24 * 60 * 60; // 29天
        weChatCacheServer.saveSessionKey(sessionKeyKey, sessionKey, expirationSeconds);
    }

    @Override  0 个用法
```

```java
    @Override  0 个用法
    public void deleteSessionKey(String sessionKeyKey) {
        log.info("删除微信SessionKey - sessionKeyKey: {}", sessionKeyKey);
        weChatCacheServer.deleteSessionKey(sessionKeyKey);
    }

    @Override  0 个用法
    public void setExpirationTime(String sessionKey, int minutes) {
        log.warn("设置微信SessionKey过期时间功能已废弃 - sessionKey: {}, 分钟: {}", sessionKey, minutes);
        // 由于使用缓存自动过期，此方法不再需要
        // 可以保留方法签名但不做具体实现，或者抛出异常
        throw new UnsupportedOperationException("该方法已废弃，SessionKey过期时间由缓存自动管理");
    }

    @Override  1 个用法
    public Map<String, Object> wechatLogin(String code) {
        log.info("开始微信登录 - Code: {}", code);

        try {
            // 调用微信接口获取 openid 和 session_key
            Map<String, String> wechatResult  = getWeChatSession(code);
            String openid = wechatResult.get("openid");
            String sessionKey = wechatResult.get("session_key");
            String expiresIn = wechatResult.get("expires_in");

            if (openid == null) {
                String errmsg = wechatResult.get("errmsg");
                log.error("微信登录失败 - Code: {}, 错误: {}", code, errmsg);
                throw new RuntimeException("微信登录失败: " + errmsg);
            }

            log.info("微信登录成功 - OpenID: {}", openid);

            // 查询用户是否存在
            WeChatAuth user = getUserByOpenid(openid);
            log.info("查询用户信息 - OpenID: {}, 用户: {}", openid, user);
            if (user == null) {
                // 创建用户
                log.info("用户不存在，创建新用户 - OpenID: {}", openid);
                createUser(openid);
                user = getUserByOpenid(openid);
            }

            // 生成 sessionKey
            String token = jwtTokenUtil.generateWeChatToken(user.getOpenId(), openid);

            // 存储 session_key 到缓存
            String sessionKeyKey = "wechat:session_key:" + openid;
            saveSessionKey(sessionKeyKey, sessionKey);

            // 返回结果
            Map<String, Object> result = MapUtils.of(
                    MapUtils.Pair.of("loginType", "wechat"),
                    MapUtils.Pair.of("token", token),
```

```
                MapUtils.Pair.of("userInfo", user),
                MapUtils.Pair.of("expiresIn", jwtTokenUtil.getExpiration())
            );
            log.info("微信登录完成 - OpenID: {}, Token生成成功", openid);
            return result;
        } catch (Exception e) {
            log.error("微信登录异常 - Code: {}", code, e);
            throw new RuntimeException("微信登录异常: " + e.getMessage());
        }
    }

    @Override 1 个用法
    public Map<String, String> getWeChatSession(String code) {
        String url = wechatConfig.getAuthUrl() +
                "?appid=" + wechatConfig.getAppId() +
                "&secret=" + wechatConfig.getAppSecret() +
                "&js_code=" + code +
                "&grant_type=authorization_code";

        log.info("调用微信登录接口 - URL: {}", url.replace(wechatConfig.getAppSecret(), replacement: "***"));
        try {
            RestTemplate restTemplate = new RestTemplate();
            String response = restTemplate.getForObject(url, String.class);
            log.info("微信接口响应: {}", response);

            Map<String, String> result = JSON.parseObject(response, Map.class);
            // 检查微信返回错误
            if (result != null && result.containsKey("errcode")) {
                String errcode = result.get("errcode");
                String errmsg = result.get("errmsg");
                if (!"0".equals(errcode)) {
                    log.error("微信接口返回错误 - ErrCode: {}, ErrMsg: {}", errcode, errmsg);
                    throw new RuntimeException("微信接口错误: " + errmsg);
                }
            }

            return result;
        } catch (Exception e) {
            log.error("微信登录接口调用失败", e);
            throw new RuntimeException("微信服务异常: " + e.getMessage());
        }
    }

    @Override 1 个用法
    public void createUser(String openid) {
        // 创建用户
        log.info("创建微信用户 - OpenID: {}", openid);

        if (openid == null) {
            throw new RuntimeException("openid不能为空");
        }

        WeChatAuth user = getUserByOpenid(openid);
        if (user == null) {
```

```
        if (user == null) {
            user = new WeChatAuth();
            user.setOpenId(openid);
            // 设置默认用户名
            user.setNickName("微信用户_" + openid.substring( beginIndex: openid.length() - 8));
            baseMapper.insert(user);
            log.info("微信用户创建成功 - OpenID: {}", openid);
        } else {
            log.info("微信用户已存在 - OpenID: {}", openid);
        }
    }

    @Override 5 个用法
    public WeChatAuth getUserByOpenid(String openid) {
        log.debug("查询微信用户 - OpenID: {}", openid);
        // 先尝试从缓存获取
        Object cachedUser = weChatCacheServer.getUserInfo(openid);
        if (cachedUser instanceof WeChatAuth) {
            log.debug("从缓存获取用户信息 - OpenID: {}", openid);
            return (WeChatAuth) cachedUser;
        }

        // 缓存中没有，从数据库查询
        WeChatAuth user = baseMapper.selectOne(new QueryWrapper<WeChatAuth>().eq( column: "open_id", openid));
        if (user != null) {
            // 存入缓存
            weChatCacheServer.saveUserInfo(openid, user);
        }

        return user;
    }

    @Override 3 个用法
    public String getUserIdFromToken(String token) { return jwtTokenUtil.getUserIdFromToken(token); }

    @Override 1 个用法
    public void updateAvatar(String openId, String avatarUrl) {
        if (openId == null) {
            throw new RuntimeException("openId不能为空");
        }

        WeChatAuth user = getUserByOpenid(openId);
        if (user == null) {
            throw new RuntimeException("用户不存在");
        }

        user.setAvatarUrl(avatarUrl);
        baseMapper.updateById(user);
    }

    @Override 1 个用法
    public WeChatAuthDTO updateUserInfo(String openId, WeChatAuthDTO user) {
        if (openId == null) {
```

```
    public WeChatAuthDTO updateUserInfo(String openId, WeChatAuthDTO user) {
        if (openId == null) {
            throw new RuntimeException("openId不能为空");
        }

        WeChatAuth weChatAuth = getUserByOpenid(openId);
        if (weChatAuth == null) {
            throw new RuntimeException("用户不存在");
        }

        weChatAuth.setNickName(user.getNickName());
        weChatAuth.setAvatarUrl(user.getAvatarUrl());
        baseMapper.updateById(weChatAuth);

        return new WeChatAuthDTO(weChatAuth.getOpenId(), weChatAuth.getAvatarUrl(), weChatAuth.getNickName());
    }
}
```

```java
package com.febuki.tool.server.service.impl;

import com.febuki.tool.server.service.WeChatCacheServer;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.cache.Cache;
import org.springframework.cache.CacheManager;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

@Slf4j
@Component
public class WeChatCacheServerImpl implements WeChatCacheServer {

    private final CacheManager wechatCacheManager;  7 个用法
    private static final String SESSION_KEY_CACHE_NAME = "wechatSessionKey";  3 个用法
    private static final String USER_INFO_CACHE_NAME = "wechatUserInfo";  3 个用法

    public WeChatCacheServerImpl(@Qualifier("wechatCacheManager") CacheManager wechatCacheManager) {
        this.wechatCacheManager = wechatCacheManager;
    }

    /**
     * 保存SessionKey到缓存
     */
    @Override  1 个用法
    public void saveSessionKey(String openid, String sessionKey, int expirationSeconds) {
        try {
            Cache cache = wechatCacheManager.getCache(SESSION_KEY_CACHE_NAME);
            if (cache != null) {
                SessionKeyInfo sessionKeyInfo = new SessionKeyInfo(sessionKey, expirationSeconds);
                cache.put(openid, sessionKeyInfo);
                log.info("微信SessionKey已缓存 - OpenID: {}, 过期时间: {}秒", openid, expirationSeconds);
            } else {
                log.error("获取微信SessionKey缓存失败");
            }
        } catch (Exception e) {
            log.error("存储微信SessionKey到缓存失败", e);
            throw new RuntimeException("存储SessionKey失败", e);
        }
    }

    /**
     * 获取SessionKey
     */
    @Override  2 个用法
    public String getSessionKey(String openid) {
        Cache cache = wechatCacheManager.getCache(SESSION_KEY_CACHE_NAME);
        if (cache == null) {
            log.error("微信SessionKey缓存未初始化");
            return null;
        }

        Cache.ValueWrapper valueWrapper = cache.get(openid);
        if (valueWrapper == null) {
```

```
        if (valueWrapper == null) {
            return null;
        }

        SessionKeyInfo sessionKeyInfo = (SessionKeyInfo) valueWrapper.get();
        if (sessionKeyInfo.isExpired()) {
            // 如果过期，删除缓存
            cache.evict(openid);
            log.info("微信SessionKey已过期 - OpenID: {}", openid);
            return null;
        }

        return sessionKeyInfo.getSessionKey();
    }

    /**
     * 删除SessionKey
     */
    @Override  1 个用法
    public void deleteSessionKey(String openid) {
        Cache cache = wechatCacheManager.getCache(SESSION_KEY_CACHE_NAME);
        if (cache != null) {
            cache.evict(openid);
            log.info("微信SessionKey已删除 - OpenID: {}", openid);
        }
    }

    /**
     * 检查SessionKey是否存在且未过期
     */
    public boolean hasValidSessionKey(String openid) {  1 个用法
        String sessionKey = getSessionKey(openid);
        return sessionKey != null && !sessionKey.isEmpty();
    }

    /**
     * 保存用户信息到缓存
     */
    public void saveUserInfo(String openid, Object userInfo) {  1 个用法
        try {
            Cache cache = wechatCacheManager.getCache(USER_INFO_CACHE_NAME);
            if (cache != null) {
                cache.put(openid, userInfo);
                log.info("微信用户信息已缓存 - OpenID: {}", openid);
            }
        } catch (Exception e) {
            log.error("存储微信用户信息到缓存失败", e);
        }
    }

    /**
     * 获取用户信息
     */
```

```java
    /**
     * 获取用户信息
     */
    public Object getUserInfo(String openid) {  1 个用法
        Cache cache = wechatCacheManager.getCache(USER_INFO_CACHE_NAME);
        if (cache == null) {
            return null;
        }

        Cache.ValueWrapper valueWrapper = cache.get(openid);
        return valueWrapper != null ? valueWrapper.get() : null;
    }

    /**
     * 删除用户信息
     */
    public void deleteUserInfo(String openid) {  0 个用法
        Cache cache = wechatCacheManager.getCache(USER_INFO_CACHE_NAME);
        if (cache != null) {
            cache.evict(openid);
            log.info("微信用户信息已删除 - OpenID: {}", openid);
        }
    }

    /**
     * SessionKey信息类
     */
    private static class SessionKeyInfo {  4 个用法
        private final String sessionKey;  2 个用法
        private final LocalDateTime expireTime;  3 个用法

        SessionKeyInfo(String sessionKey, int expirationSeconds) {  1 个用法
            this.sessionKey = sessionKey;
            this.expireTime = LocalDateTime.now().plusSeconds(expirationSeconds);
        }

        String getSessionKey() { return sessionKey; }

        boolean isExpired() { return LocalDateTime.now().isAfter(expireTime); }

        long getRemainingSeconds() { return LocalDateTime.now().until(expireTime, ChronoUnit.SECONDS); }
    }
}
```

```java
package com.febuki.tool.server.controller;

import ...

@Slf4j
@RestController
@RequestMapping("/api/auth")
@Tag(name = "微信授权", description = "微信授权接口")
public class WeChatAuthController {

    private final String uploadDir;  2 个用法

    private final String accessUrl;  2 个用法

    private final WeChatAuthServer wechatAuthServer;  9 个用法

    public WeChatAuthController(
            @Value("${file.upload-dir}") String uploadDir,
            @Value("${file.access-url}") String accessUrl,
            WeChatAuthServer wechatAuthServer
    ) {
        this.uploadDir = uploadDir;
        this.accessUrl = accessUrl;
        this.wechatAuthServer = wechatAuthServer;
    }

    // 微信登录
    @PostMapping("/wechat-login")
    @Operation(summary = "微信登录", description = "微信登录")
    public ResponseEntity<?> wechatLogin(
            @Parameter(
                    description = "",
                    required = true
            )
            @RequestBody Map<String, String> params
    ) {
        String code = params.get("code");
        log.info("微信登录: code={}", code);
        if (code == null) {
            log.error("微信登录失败: code 不能为空");
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.BAD_REQUEST.value()),
                            MapUtils.Pair.of("error", "code 不能为空")
                    )
            );
        }

        try {
            Map<String, Object> result = wechatAuthServer.wechatLogin(code);
            return ResponseEntity.status(HttpStatus.OK).body(result);
        } catch (Exception e) {
            log.error("微信登录失败", e);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.INTERNAL_SERVER_ERROR.value()),
                            MapUtils.Pair.of("message", "登录失败"),
                            MapUtils.Pair.of("error", e.getMessage())
```

```
                        MapUtils.Pair.of("message", "登录失败"),
                        MapUtils.Pair.of("error", e.getMessage())
                )
        );
    }
}

// 验证 token
@PostMapping("/validate-token")
@Operation(summary = "验证 token", description = "验证 token")
public ResponseEntity<?> validateToken(
        @Parameter(
                description = "token",
                required = true
        )
        @RequestHeader("Authorization")
        String token
) {
    log.info("验证 token: token={}", token);
    if (token == null) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
                MapUtils.of(
                        MapUtils.Pair.of("code", HttpStatus.BAD_REQUEST.value()),
                        MapUtils.Pair.of("error", "token 不能为空")
                )
        );
    }

    try {
        boolean isValid = wechatAuthServer.validateToken(token);
        log.info("验证 token 结果: isValid={}", isValid);

        Map<String, Object> result = new HashMap<>();
        result.put("isValid", isValid);
        if (isValid) {
            String userId = wechatAuthServer.getUserIdFromToken(token);
            result.put("userId", userId);
        }
        result.put("code", HttpStatus.OK.value());
        return ResponseEntity.status(HttpStatus.OK).body(result);
    } catch (Exception e) {
        log.error("验证 token 失败", e);
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(
                MapUtils.of(
                        MapUtils.Pair.of("code", HttpStatus.UNAUTHORIZED.value()),
                        MapUtils.Pair.of("message", "token 验证失败"),
                        MapUtils.Pair.of("error", e.getMessage())
                )
        );
    }
}

// 上传头像
@PostMapping("/upload-avatar")
@Operation(summary = "上传头像", description = "上传头像")
```

```java
@Operation(summary = "上传头像", description = "上传头像")
public ResponseEntity<?> uploadAvatar(
        @Parameter(
                description = "头像",
                required = true
        )
        @RequestParam("avatar")
        MultipartFile file,
        @Parameter(
                description = "request",
                required = true
        )
        HttpServletRequest request
) throws IOException {
    try {
        // 验证 token
        String token = request.getHeader( s: "Authorization");
        log.info("上传头像: token={}", token);
        if (!wechatAuthServer.validateToken(token)) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.UNAUTHORIZED.value()),
                            MapUtils.Pair.of("message", "token 验证失败"),
                            MapUtils.Pair.of("error", "token 无效")
                    )
            );
        }

        String openId = wechatAuthServer.getUserIdFromToken(token);
        log.info("上传头像: openId={}", openId);

        // 验证文件
        if (file == null || file.isEmpty()) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.BAD_REQUEST.value()),
                            MapUtils.Pair.of("message", "文件不能为空")
                    )
            );
        }

        // 验证文件类型
        String contentType = file.getContentType();
        if (contentType != null && !contentType.startsWith("image/")) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.BAD_REQUEST.value()),
                            MapUtils.Pair.of("message", "只能上传图片"),
                            MapUtils.Pair.of("error", "文件类型错误")
                    )
            );
        }

        // 生成文件名
        String originalFilename = file.getOriginalFilename();
        String fileExtension = originalFilename.substring(originalFilename.lastIndexOf( str: "."));
```

```
            String fileName = "avatar_" + openId + "_" + System.currentTimeMillis() + fileExtension;

            // 创建目录
            File uploadPath = new File(uploadDir);
            if (!uploadPath.exists()) {
                uploadPath.mkdirs();
            }

            // 保存文件
            File destFile = new File(uploadPath, fileName);
            file.transferTo(destFile);

            // 生成访问地址
            String avatarUrl = accessUrl + fileName;

            // 更新用户头像
            wechatAuthServer.updateAvatar(openId, avatarUrl);

            return ResponseEntity.status(HttpStatus.OK).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.OK.value()),
                            MapUtils.Pair.of("message", "上传成功"),
                            MapUtils.Pair.of("avatarUrl", avatarUrl)
                    )
            );
        } catch (Exception e) {
            log.error("上传头像失败", e);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.INTERNAL_SERVER_ERROR.value()),
                            MapUtils.Pair.of("message", "上传头像失败"),
                            MapUtils.Pair.of("error", e.getMessage())
                    )
            );
        }
    }

    // 更新用户信息
    @PostMapping("/update-info")
    @Operation(summary = "更新用户信息", description = "更新用户信息")
    public ResponseEntity<?> updateInfo(
            @Parameter(
                    description = "request",
                    required = true
            )
            @RequestBody WeChatAuthDTO weChatAuthDTO,
            @Parameter(
                    description = "httpRequest",
                    required = true
            )
            HttpServletRequest request
    ) {
        try {
            String token = request.getHeader( s: "Authorization");
```

```
    // 更新用户信息
    @PostMapping("/update-info")
    @Operation(summary = "更新用户信息", description = "更新用户信息")
    public ResponseEntity<?> updateInfo(
            @Parameter(
                    description = "request",
                    required = true
            )
            @RequestBody WeChatAuthDTO weChatAuthDTO,
            @Parameter(
                    description = "httpRequest",
                    required = true
            )
            HttpServletRequest request
    ) {
        try {
            String token = request.getHeader( s: "Authorization");
            String openId = wechatAuthServer.getUserIdFromToken(token);

            // 更新用户信息
            WeChatAuthDTO user = wechatAuthServer.updateUserInfo(openId, weChatAuthDTO);

            return ResponseEntity.status(HttpStatus.OK).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.OK.value()),
                            MapUtils.Pair.of("message", "更新成功"),
                            MapUtils.Pair.of("userInfo", user)
                    )
            );
        } catch (Exception e) {
            log.error("更新用户信息失败", e);
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                    MapUtils.of(
                            MapUtils.Pair.of("code", HttpStatus.INTERNAL_SERVER_ERROR.value()),
                            MapUtils.Pair.of("message", "更新用户信息失败"),
                            MapUtils.Pair.of("error", e.getMessage())
                    )
            );
        }
    }
}
```

第 5 章 项目的测试

5.1、系统管理子系统

5.1.1、口令管理

- 测试内容：

主要验证在线旅游预订平台的开发与推广系统中登录与口令管理功能是否满足设计要求，包括：

- 手机号 + 短信验证码登录；
- 手机号 + 密码登录；

- 对手机号格式的前端校验；
- 错误账号/密码时系统的提示与稳定性。

- 测试数据：
  - 多组 11 位合法手机号码；
  - 非 11 位手机号、含非数字字符的手机号；
  - 正确验证码、错误验证码；
  - 正确密码、错误密码

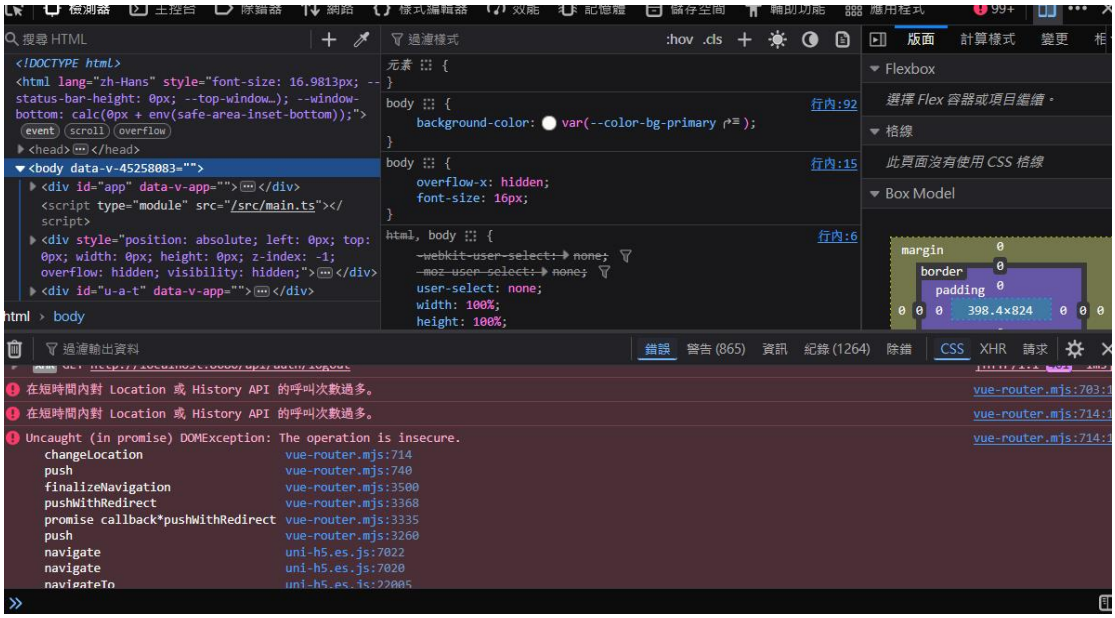
- 测试方法及步骤：
  - 步骤 1：访问后台登录界面和用户端的登录界面，默认为“手机号验证码登录”模式；
  - 步骤 2（验证码登录）：
    - 依次输入不合法手机号（位数不足/超出 11 位、包含非数字字符），观察“获取验证码”按钮是否被禁用以及错误提示；
    - 输入合法手机号并获取验证码，在有效时间内输入正确验证码，点击“验证码登录/注册”按钮，观察是否正常跳转到系统首页；
  - 步骤 3（密码登录）：
    - 点击“密码登录”切换登录方式；
    - 输入正确手机号 + 错误密码，点击“密码登录”按钮，观察页面表现、错误提示及是否出现异常刷新；
  - 步骤 4：记录各用例的预期输出与实际输出，判断是否一致。

- 测试结果：以上各口令的登录功能均运行正确，满足功能要求。

输入 （账号、密码）	预期输出	测试输出	结果分析	原因分析
手机号：合法 11 位数字；验证码：正确	正确登录首页	正确登录首页	一致	
手机号：非 11 位或包含非数字字符	“获取验证码”按钮置灰不可点击；	实际按钮不可点击，无法发送验证码，前端限制无	一致	

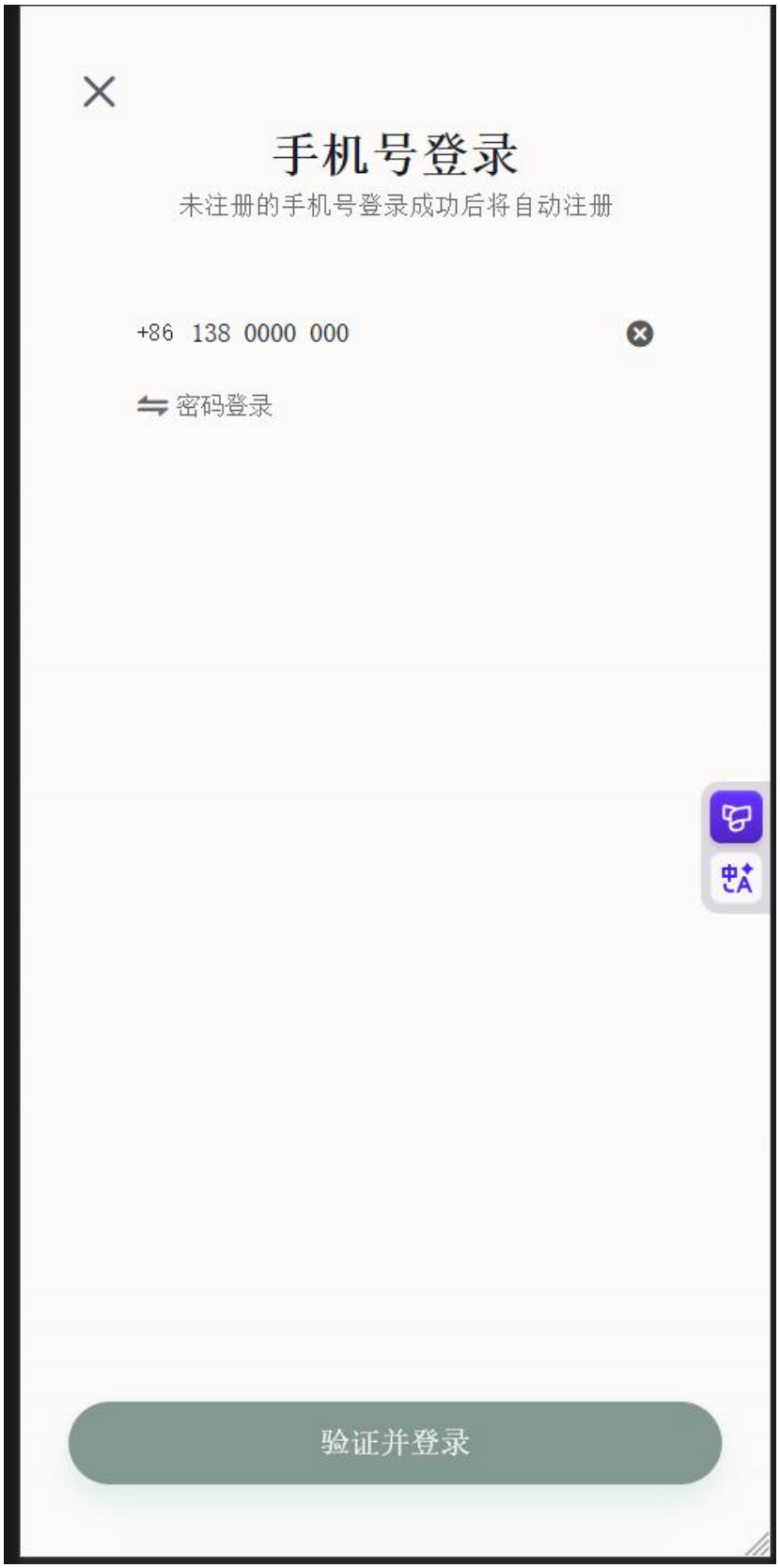
		法输入超过 11 位数字		
手机号：合法 11 位数字; 验证码：不正确	提示请输入正确的验证码	实际：用户端提示验证码错误或已过期，后台未报错直接登录进去了	不一致	可能是由于假数据测试时的判断未删除导致前端认为后端报错也是正确的
手机号：合法 11 位数字; 密码错误；	提示账号或密码错误	提交后用户端页面出现异常：登录页反复闪烁/类似不断重载；控制台瞬间输出 2000+ 条 Vue Router 警告信息；后台能够正常提示	不一致	怀疑存在登录失败触发全局登出 + 路由循环跳转的问题；
手机号：合法 11 位数字; 密码正确；	正确登录首页	用户端依旧闪烁； 后台能正常登录		怀疑存在登录失败触发全局登出 + 路由循环跳转的问题；

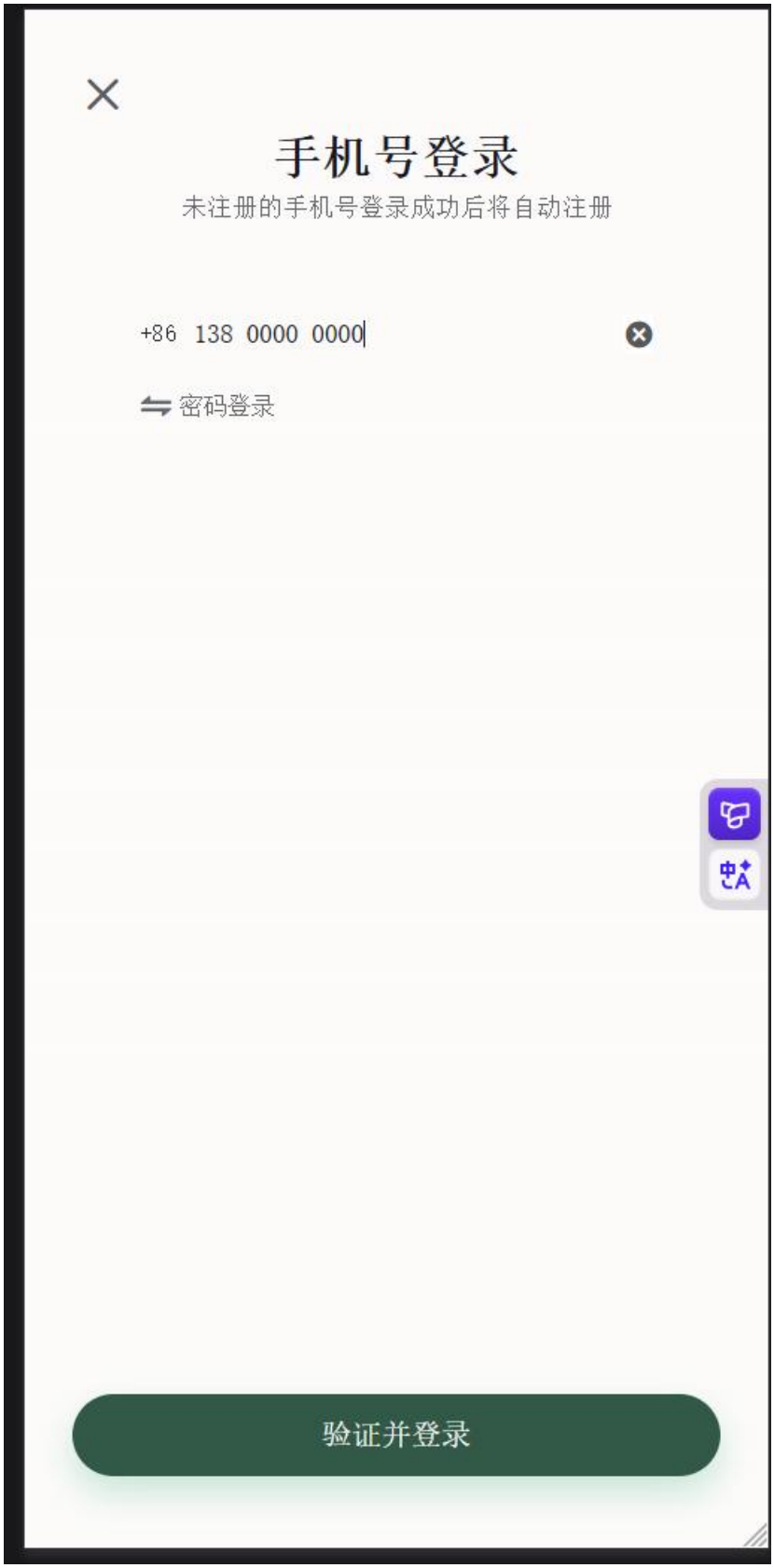
过程截图：


檢測器
主控台
除錯器
網路
樣式編輯器
效能
記憶體
儲存空間
輔助功能
應用程式
99+
搜尋 HTML
<!DOCTYPE html>
<html lang="zh-Hans" style="font-size: 16.9813px; --status-bar-height: 0px; --top-window…); --window-bottom: calc(0px + env(safe-area-inset-bottom));">
event
scroll
overflow
<head> </head>
<body data-v-45258083="">

<script type="module" src="/src/main.ts"></script>

html › body
過濾樣式
:hov .cls
元素 {
}
body {
background-color: var(--color-bg-primary);
行內:92
}
body {
overflow-x: hidden;
font-size: 16px;
行內:15
}
html, body {
-webkit-user-select: none;
-moz-user-select: none;
user-select: none;
width: 100%;
height: 100%;
行內:6
版面
計算樣式
變更
Flexbox
選擇 Flex 容器或項目繼續。
格線
此頁面沒有使用 CSS 格線
Box Model
margin 0
border 0
padding 0
0 0 0 398.4x824 0 0 0
過濾輸出資料
錯誤
警告 (865)
資訊
紀錄 (1264)
除錯
CSS
XHR
請求
在短時間內對 Location 或 History API 的呼叫次數過多。
vue-router.mjs:703:1
在短時間內對 Location 或 History API 的呼叫次數過多。
vue-router.mjs:714:1
Uncaught (in promise) DOMException: The operation is insecure.
vue-router.mjs:714:1
changeLocation vue-router.mjs:714
push vue-router.mjs:740
finalizeNavigation vue-router.mjs:3500
pushWithRedirect vue-router.mjs:3368
promise callback*pushWithRedirect vue-router.mjs:3335
push vue-router.mjs:3260
navigate uni-h5.es.js:7022
navigate uni-h5.es.js:7020
navigateTo uni-h5.es.js:22005


手机号登录
未注册的手机号登录成功后将自动注册
+86 138 0000 000
密码登录
验证并登录


手机号登录
未注册的手机号登录成功后将自动注册
+86 138 0000 0000
密码登录
验证并登录


Hello 用户_0000
welcome to trip
Search
select your next trip
交通
附近
推荐
酒店
夕阳红团
4.9
含机酒服务 · 四晚五星酒店 · 私导陪同
含接送
私导团
双人房
see more

登录商家中心

默认手机号验证码登录，可切换密码方式

手机号登录 密码登录

手机号

13888888888

验证码

短信验证码 56s

验证码登录/注册

验证码发送成功

登录商家中心

默认手机号验证码登录，可切换密码方式

手机号登录 密码登录

手机号

1388888888

验证码

短信验证码 获取验证码

验证码登录/注册

登录商家中心

默认手机号验证码登录，可切换密码方式

手机号登录 密码登录

手机号

13888888888

密码

••••••

密码登录仅限管理员或已设置密码的商家

密码登录

账号或密码错误

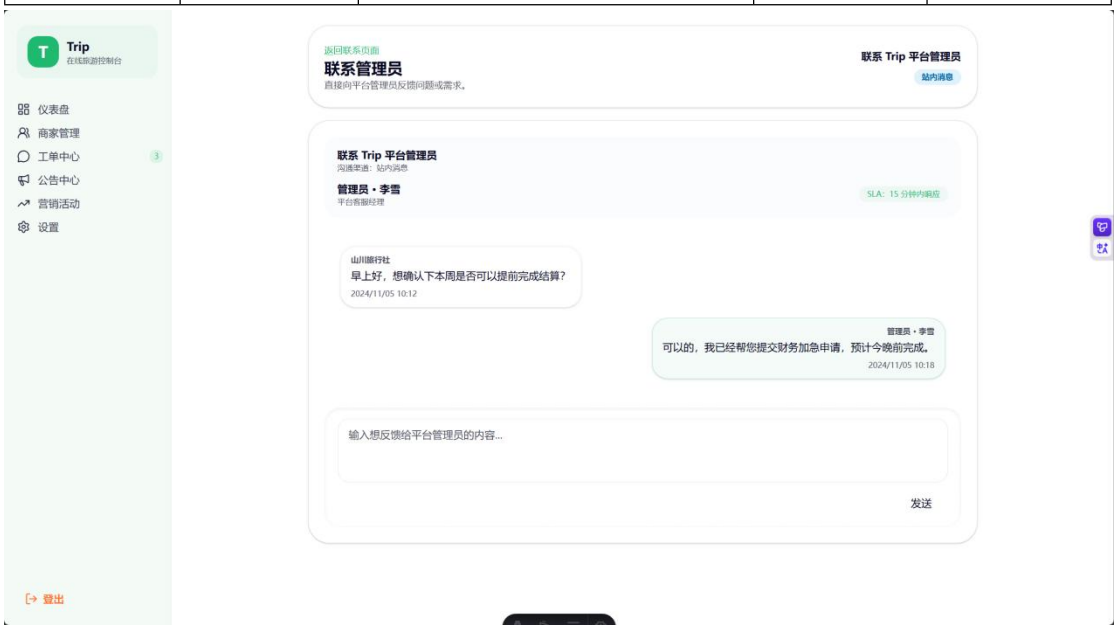


5.1.2 Web 管理

- 测试内容： 检查 Web 相关内容功能
- 测试数据： Web 相关内容
- 测试方法及步骤： 点击 Web 相关按钮功能，检查是否显示错误
- 测试结果：

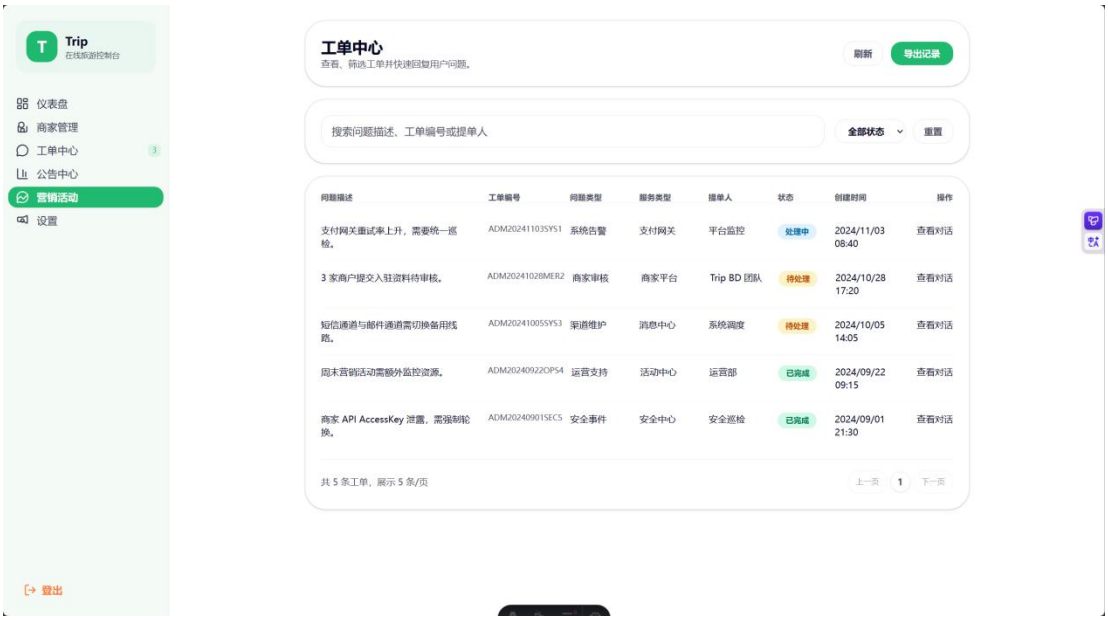
输入	预期输出	测试输出	结果分析	原因分析
	转跳正确	管理端侧边栏点击工单中心再点击设置跳转错误	不一致	配置错误
	显示正确	管理端工单中心角标显示错误	不一致	配置错误

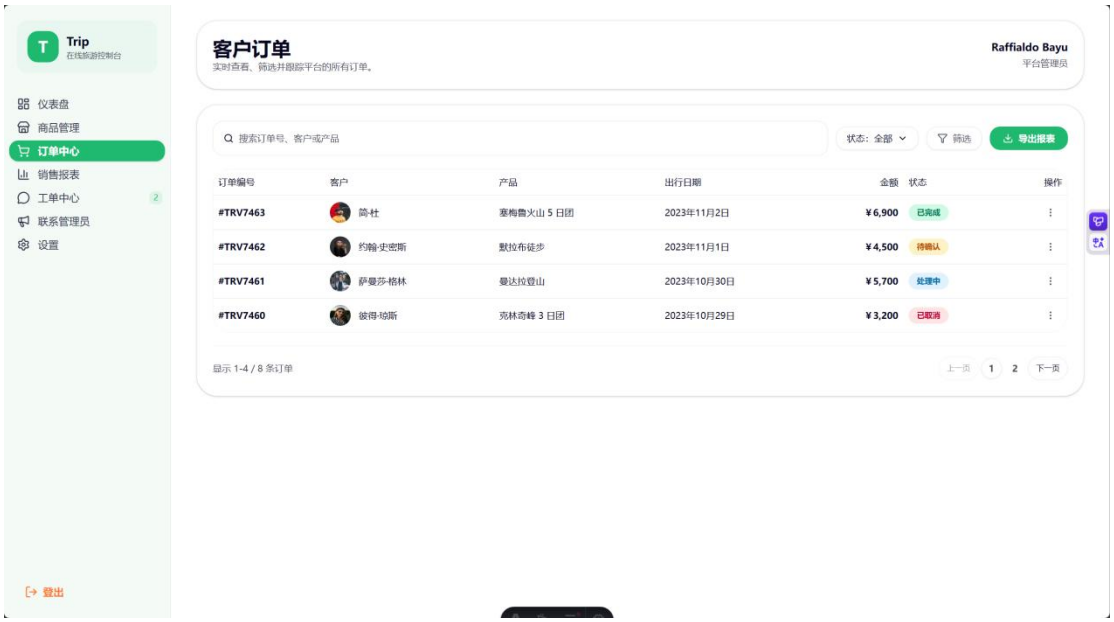
	跳转正确	管理后台侧边栏点击工单中心按钮跳转错误	不一致	配置错误
	显示正确	商家端侧边栏工单中心角标错误	不一致	配置错误
	显示正确	商家端点击联系管理员侧边栏图标显示错误	不一致	配置错误
	跳转正确	商家端点击两次联系管理员跳转错误	不一致	配置错误
	显示正确	商家端工单中心点击工单对话侧边栏显示错误	不一致	配置错误
	跳转正确	商家端工单中心点击工单对话再点击工单中心跳转错误	不一致	配置错误
	跳转正确	商家端工单中心点击工单对话再点击联系管理员跳转错误	不一致	配置错误



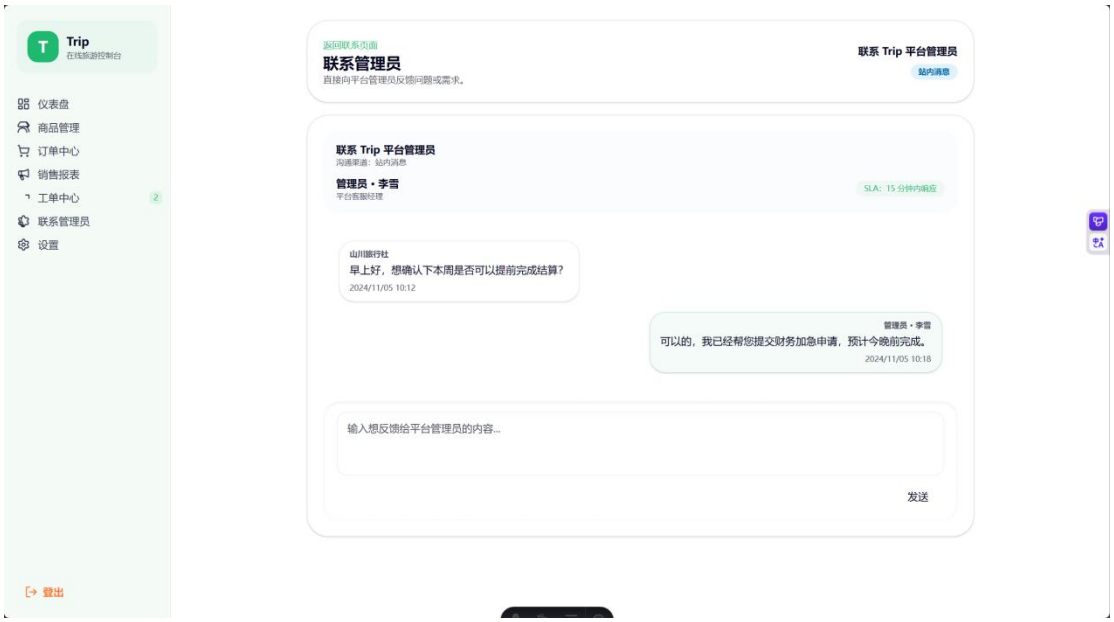

T
Trip
在线旅游控制台
仪表盘
商家管理
工单中心
3
公告中心
营销活动
设置

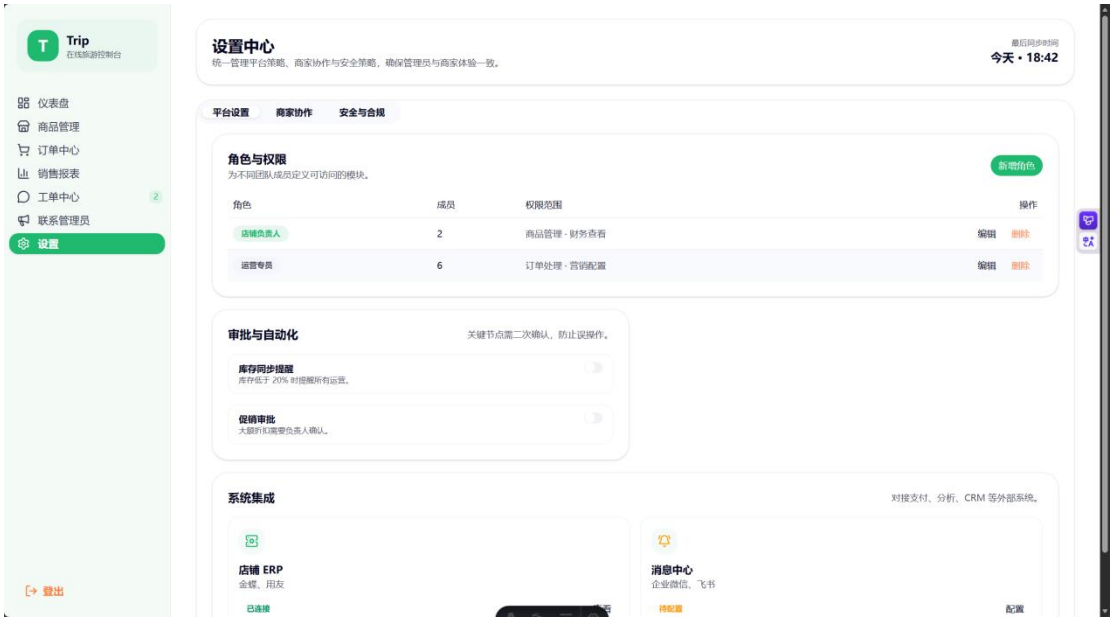

仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置

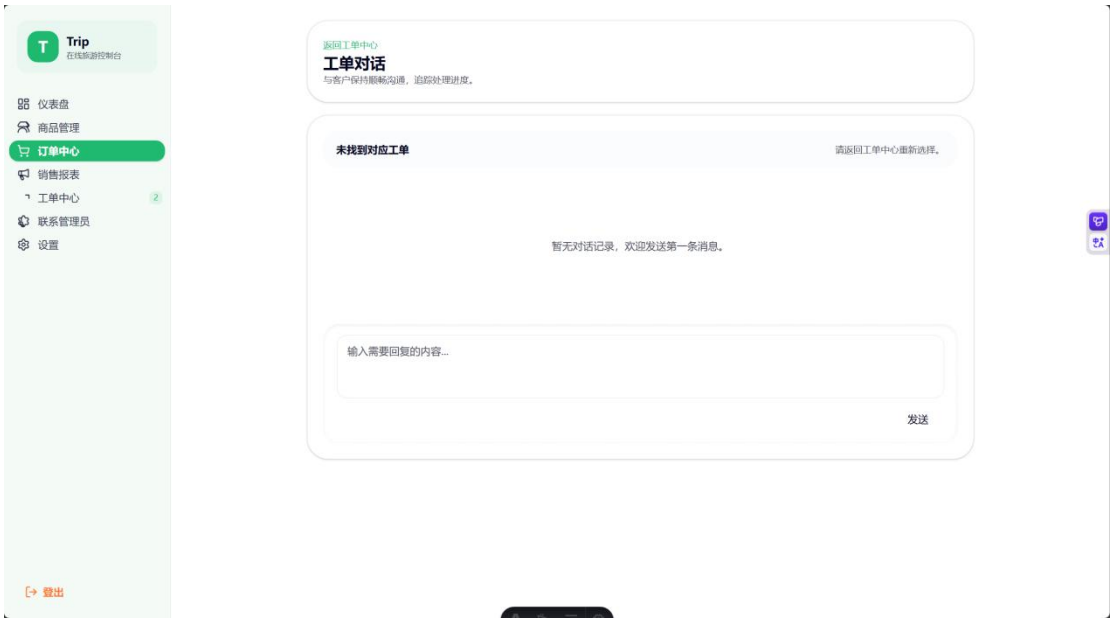

T
Trip
在线旅游控制台
仪表盘
商家管理
工单中心
3
公告中心
营销活动
设置
登出
工单中心
查看、筛选工单并快速回复用户问题。
刷新
导出记录
搜索问题描述、工单编号或提单人
全部状态
重置
问题描述
工单编号
问题类型
服务类型
提单人
状态
创建时间
操作
支付网关重试率上升，需要统一巡检。
ADM20241103SYS1
系统告警
支付网关
平台监控
处理中
2024/11/03 08:40
查看对话
3 家商户提交入驻资料待审核。
ADM20241028MER2
商家审核
商家平台
Trip BD 团队
待处理
2024/10/28 17:20
查看对话
短信通道与邮件通道需切换备用线路。
ADM20241005SYS3
渠道维护
消息中心
系统调度
待处理
2024/10/05 14:05
查看对话
周末营销活动需额外监控资源。
ADM20240922OPS4
运营支持
活动中心
运营部
已完成
2024/09/22 09:15
查看对话
商家 API AccessKey 泄露，需强制轮换。
ADM20240901SEC5
安全事件
安全中心
安全巡检
已完成
2024/09/01 21:30
查看对话
共 5 条工单，展示 5 条/页
上一页
1
下一页

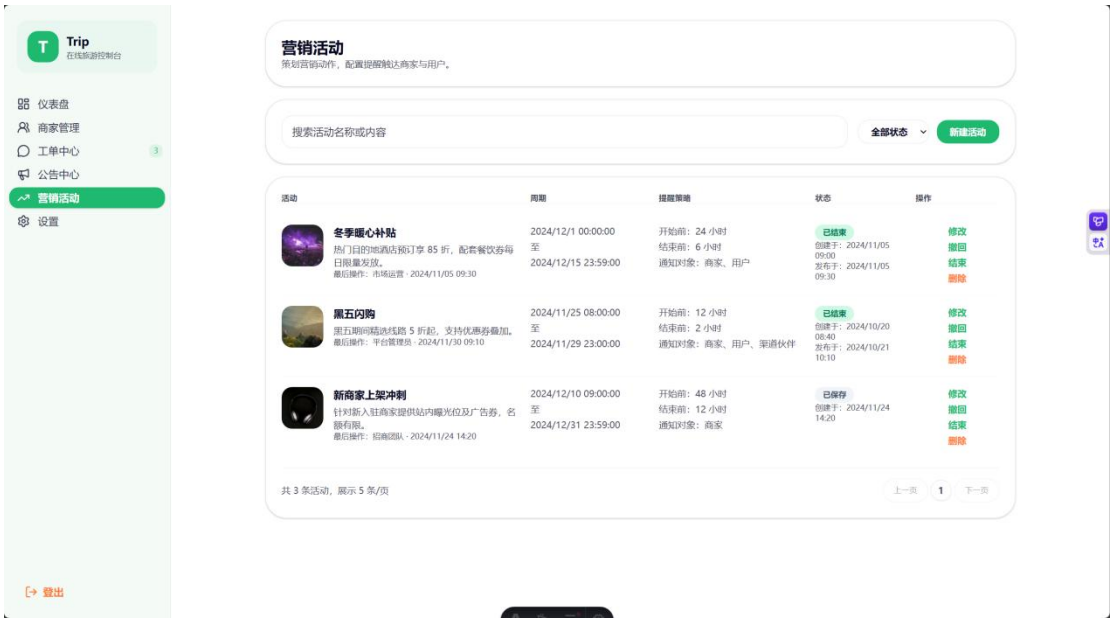

Trip
在线旅游控制台
仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置
登出
客户订单
实时查看、筛选并跟踪平台的所有订单。
Raffialdo Bayu
平台管理员
搜索订单号、客户或产品
状态：全部
筛选
导出报表
订单编号
客户
产品
出行日期
金额
状态
操作
#TRV7463
简·杜
塞梅鲁火山 5 日团
2023年11月2日
¥6,900
已完成
#TRV7462
约翰·史密斯
默拉布徒步
2023年11月1日
¥4,500
待确认
#TRV7461
萨曼莎·格林
曼达拉登山
2023年10月30日
¥5,700
处理中
#TRV7460
彼得·琼斯
克林奇峰 3 日团
2023年10月29日
¥3,200
已取消
显示 1-4 / 8 条订单
上一页
1
2
下一页

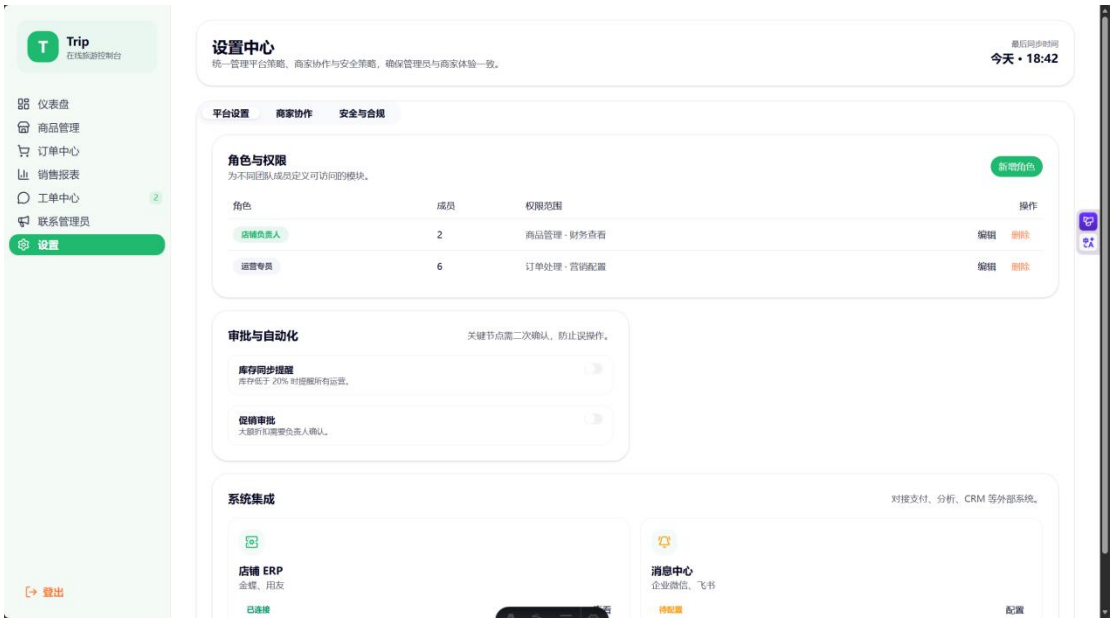

仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置


Trip
在线旅游控制台
仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置
登出
返回联系中心
联系管理员
直接向平台管理员反馈问题或需求。
联系 Trip 平台管理员
站内消息
联系 Trip 平台管理员
沟通渠道：站内消息
管理员 · 李雪
平台客服经理
SLA：15 分钟内响应
山川旅行社
早上好，想确认下本周是否可以提前完成结算？
2024/11/05 10:12
管理员 · 李雪
可以的，我已经帮您提交财务加急申请，预计今晚前完成。
2024/11/05 10:18
输入想反馈给平台管理员的内容...
发送


Trip
在线旅游控制台
仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置
登出
设置中心
统一管理平台策略、商家协作与安全策略，确保管理员与商家体验一致。
最后同步时间
今天 · 18:42
平台设置
商家协作
安全与合规
角色与权限
为不同团队成员定义可访问的模块。
新增角色
角色
成员
权限范围
操作
店铺负责人
2
商品管理 · 财务查看
编辑
删除
运营专员
6
订单处理 · 营销配置
编辑
删除
审批与自动化
关键节点需二次确认，防止误操作。
库存同步提醒
库存低于 20% 时提醒所有运营。
促销审批
大额折扣需要负责人确认。
系统集成
对接支付、分析、CRM 等外部系统。
店铺 ERP
金蝶、用友
已连接
消息中心
企业微信、飞书
待配置
配置


Trip
在线旅游控制台
仪表盘
商品管理
订单中心
销售报表
工单中心
2
联系管理员
设置
登出
返回工单中心
工单对话
与客户保持顺畅沟通，追踪处理进度。
未找到对应工单
请返回工单中心重新选择。
暂无对话记录，欢迎发送第一条消息。
输入需要回复的内容...
发送


Trip
在线旅游控制台
仪表盘
商家管理
工单中心
3
公告中心
营销活动
设置
登出
营销活动
策划营销动作，配置提醒触达商家与用户。
搜索活动名称或内容
全部状态
新建活动
活动
周期
提醒策略
状态
操作
冬季暖心补贴
热门目的地酒店预订享 85 折，配套餐饮券每日限量发放。
最后操作：市场运营 · 2024/11/05 09:30
2024/12/1 00:00:00
至
2024/12/15 23:59:00
开始前：24 小时
结束前：6 小时
通知对象：商家、用户
已结束
创建于：2024/11/05 09:00
发布于：2024/11/05 09:30
修改
撤回
结束
删除
黑五闪购
黑五期间精选线路 5 折起，支持优惠券叠加。
最后操作：平台管理员 · 2024/11/30 09:10
2024/11/25 08:00:00
至
2024/11/29 23:00:00
开始前：12 小时
结束前：2 小时
通知对象：商家、用户、渠道伙伴
已结束
创建于：2024/10/20 08:40
发布于：2024/10/21 10:10
修改
撤回
结束
删除
新商家上架冲刺
针对新入驻商家提供站内曝光位及广告券，名额有限。
最后操作：招商团队 · 2024/11/24 14:20
2024/12/10 09:00:00
至
2024/12/31 23:59:00
开始前：48 小时
结束前：12 小时
通知对象：商家
已保存
创建于：2024/11/24 14:20
修改
撤回
结束
删除
共 3 条活动，展示 5 条/页
上一页
1
下一页

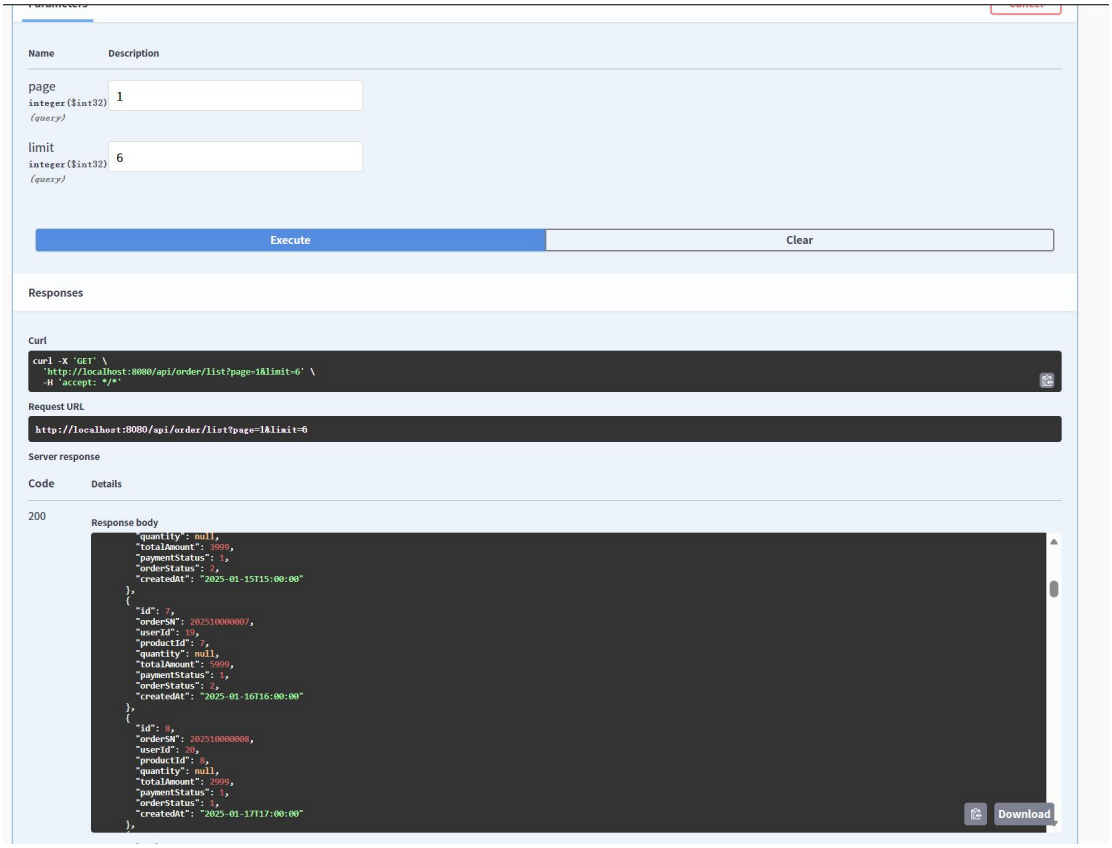


5.2 订单管理

5.2.1 订单列表

- 测试内容： 验证分页获取订单列表功能
- 测试数据： 页数，订单数量
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单列表	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	
订单列表	用户能看到自己的订单	没有联表查询，	不一致	



5.1.2.2 订单详情

- 测试内容：验证 id 获取订单功能
- 测试数据： 订单 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单 id	订单数据	订单数据	一致	

5.2.3 支付订单

- 测试内容： 验证支付操作功能
- 测试数据： 订单 id
- 测试方法及步骤：在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单 id	完成订单	完成订单	一致	

5.2.4 确认收货/消费

- 测试内容： 验证订单状态从'未完成'改为'待评价'功能
- 测试数据： 订单 id
- 测试方法及步骤：在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单 id	完成订单	完成订单	一致	

5.2.5 完成订单

- 测试内容： 验证订单状态从'待评价'改为'已完成'功能
- 测试数据： 订单 id
- 测试方法及步骤：在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单 id	完成订单	完成订单	一致	

5.2.6 添加订单

- 测试内容： 验证添加订单功能
- 测试数据： 用户 id，商品 id，商品数量
- 测试方法及步骤：在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
用户 id， 商品 id， 商品数量	添加订单	添加订单	一致	

5.3 订单项接口

5.3.1 删除订单项

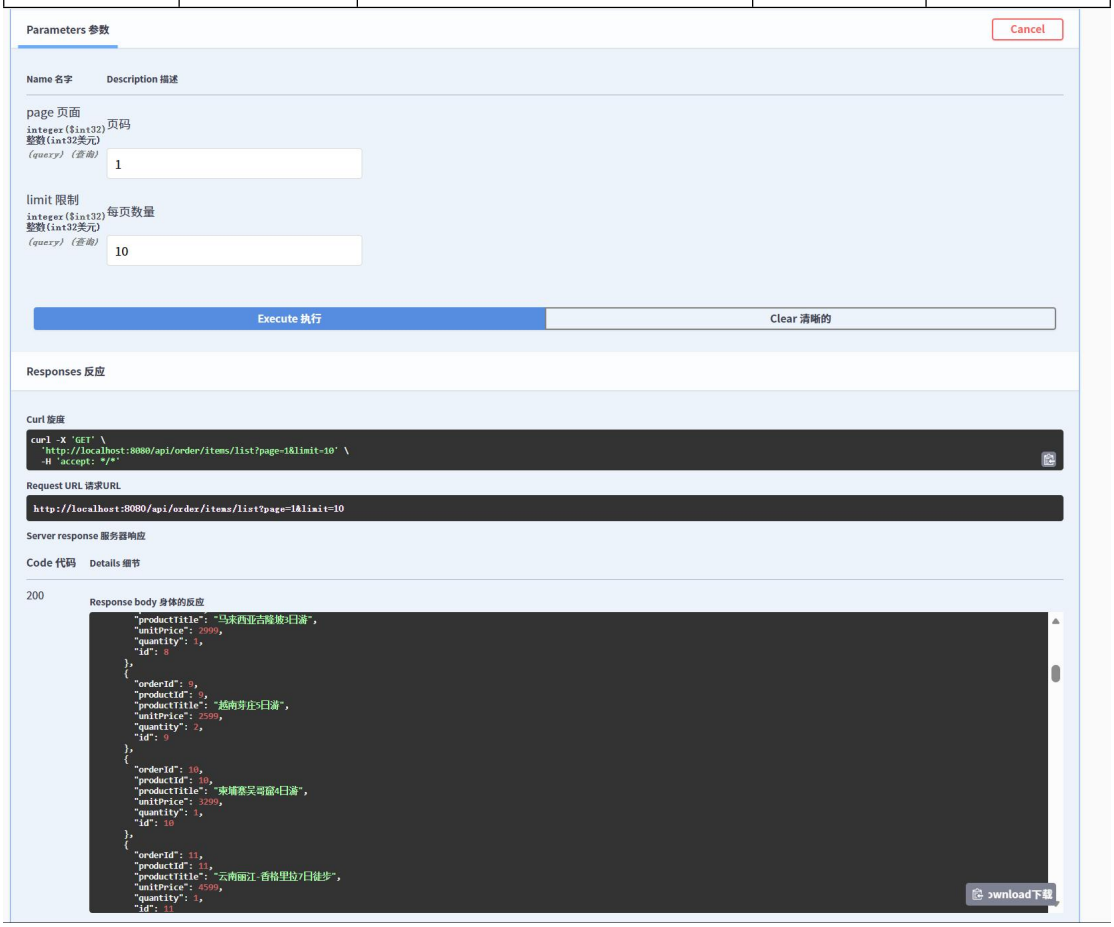
- 测试内容： 验证根据 id 删除订单项功能
- 测试数据：订单项 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各项数据均能顺利录入编辑，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单项 id	删除订单	删除订单	一致	

5.3.2 订单项列表

- 测试内容：验证订单项列表功能
- 测试数据： 订单项
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的均能动态输出，且对比正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单项	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	



5.3.3 订单项详情

- 测试内容：验证获取单个订单项详情的功能
- 测试数据： 订单项 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
订单项 id	查看详情	查看详情	一致	

5.4　短信验证码服务

5.4.1 发送短信验证码

- 测试内容：验证发送短信验证码功能
- 测试数据：　手机号
- 测试方法及步骤：　在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
手机号	发送短信验证码	发送短信验证码	一致	

5.4.2　短信登录

- 测试内容：　验证短信登录功能
- 测试数据：　手机号，验证码
- 测试方法及步骤：　在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
手机号，验证码	成功登录	成功登录	一致	

5.5 火车服务

5.5.1　获取列车站到站时刻表

- 测试内容：　验证列车站到站时刻表获取功能
- 测试数据：　查询类型，出发站，到达站，日期，是否预定，出发时间范围
- 测试方法及步骤：　在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
查询类型，出发站，到达站，日期，是否预定，出发	获取列车站到站时刻表	获取列车站到站时刻表	一致	

时间范围				

5.6 景点服务

5.6.1 获取景点信息

- 测试内容： 验证获取景点信息功能
- 测试数据： 省，城市，地点
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
省，城市，地点	获取景点信息	获取景点信息	一致	

5.7 旅客管理

5.7.1 删除游客

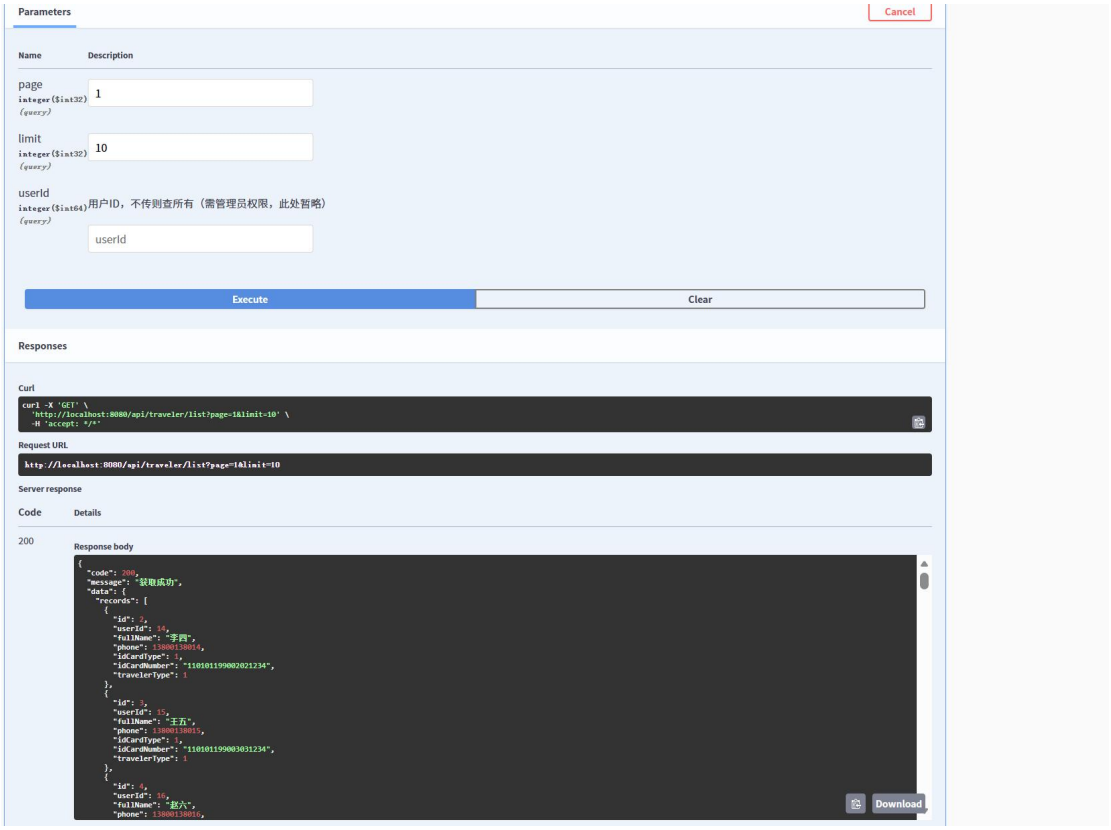
- 测试内容： 验证根据 ID 删除游客功能
- 测试数据： 游客 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
游客 id	删除游客	删除游客	一致	

5.7.2 游客列表

- 测试内容： 验证分页获取游客列表功能
- 测试数据： 页数，游客数量
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
页数,游客数量	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	



5.7.3 游客详情

- 测试内容： 验证根据 ID 获取游客详情功能
- 测试数据： 游客 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
游客 id	查看数据	查看数据	一致	

5.7.4 添加游客

- 测试内容： 验证添加游客功能
- 测试数据： 游客 id，姓名，手机号，身份证类型，身份证号，游客类型
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
游客 id，姓名，手机号，身份证	添加数据	添加数据	一致	

类型，身份证号，游客类型				

5.8 商户管理

5.8.1 删除商户

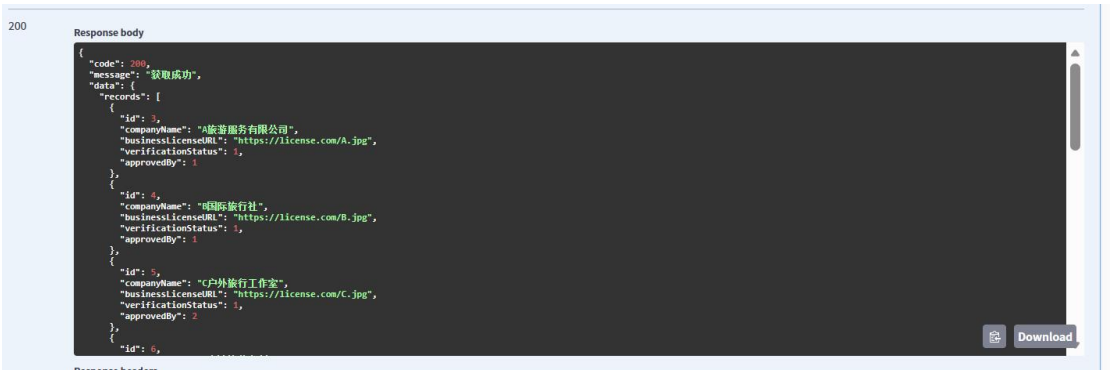
- 测试内容： 验证删除商户功能
- 测试数据： 商户 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商户 id	删除商户	删除商户	一致	

5.8.2 商户列表

- 测试内容： 验证商户列表获取功能
- 测试数据： 页数，商户数
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
页数，商户数	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	



5.8.3 商户详细

- 测试内容： 验证获取商户详情功能
- 测试数据： 商户 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商户 id	查看数据	查看数据	一致	

5.8.4 添加商户

- 测试内容： 验证添加商户功能
- 测试数据： 商户 id ， 商家名称，验证状态，审核人员
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商户 id ，商家名称，验证状态，审核人员	添加商户	添加商户	一致	

5.8.5 更新商户

- 测试内容： 验证更新商户信息功能
- 测试数据： 商户 id ， 商家名称，验证状态，审核人员
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商户 id ，商家名称，验证状态，审核人员	更新商户信息	更新商户信息	一致	

5.9 商品接口

5.9.1 下架商品

- 测试内容： 验证根据 ID 下架商品功能

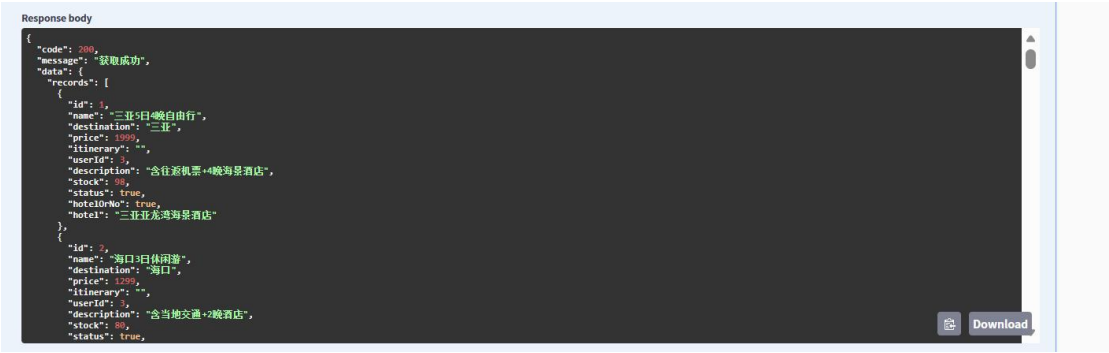
- 测试数据： 商品 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商品 id	下架商品	下架商品	一致	

5.9.2 商品列表

- 测试内容： 验证分页获取商品列表功能
- 测试数据： 页数，商品数
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
页数，商品数	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	



5.9.3 商品详细

- 测试内容： 验证根据 ID 获取商品详情功能
- 测试数据： 商品 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商品 id	成功获取商品详情	成功获取商品详情	一致	

5.9.4 添加商品

- 测试内容： 验证添加商品功能
- 测试数据： 商品名称，商品所在地，商品价格，商品标识，商品描述，是

否为酒店

- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商品名称，商品所在地，商品价格，商品标识，商品描述，是否为酒店	成功添加商品	成功添加商品	一致	

5.9.5 更新商品

- 测试内容： 验证更新商品信息功能
- 测试数据： 商品名称，商品所在地，商品价格，商品标识，商品描述，是否为酒店
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
商品名称，商品所在地，商品价格，商品标识，商品描述，是否为酒店	成功更新商品信息	成功更新商品信息	一致	

5.10 微信授权

5.10.1 微信登录

- 测试内容： 验证微信登录功能
- 测试数据： 无

- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
	成功登录微信	成功登录微信	一致	

5.10.2 验证 token

- 测试内容： 验证验证 token 功能
- 测试数据： token 令牌
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
Token 令牌	验证 Token 令牌成功	验证 Token 令牌成功	一致	

5.10.3 上传头像

- 测试内容： 验证上传头像功能
- 测试数据： 微信头像
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
微信头像	头像上传成功	头像上传成功	一致	

5.10.4 更新用户信息

- 测试内容： 验证更新用户信息功能
- 测试数据： 微信开放 ID，微信头像，微信昵称
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
微信开放 ID,微信头像,微信昵	成功更新用户信息	成功更新用户信息	一致	

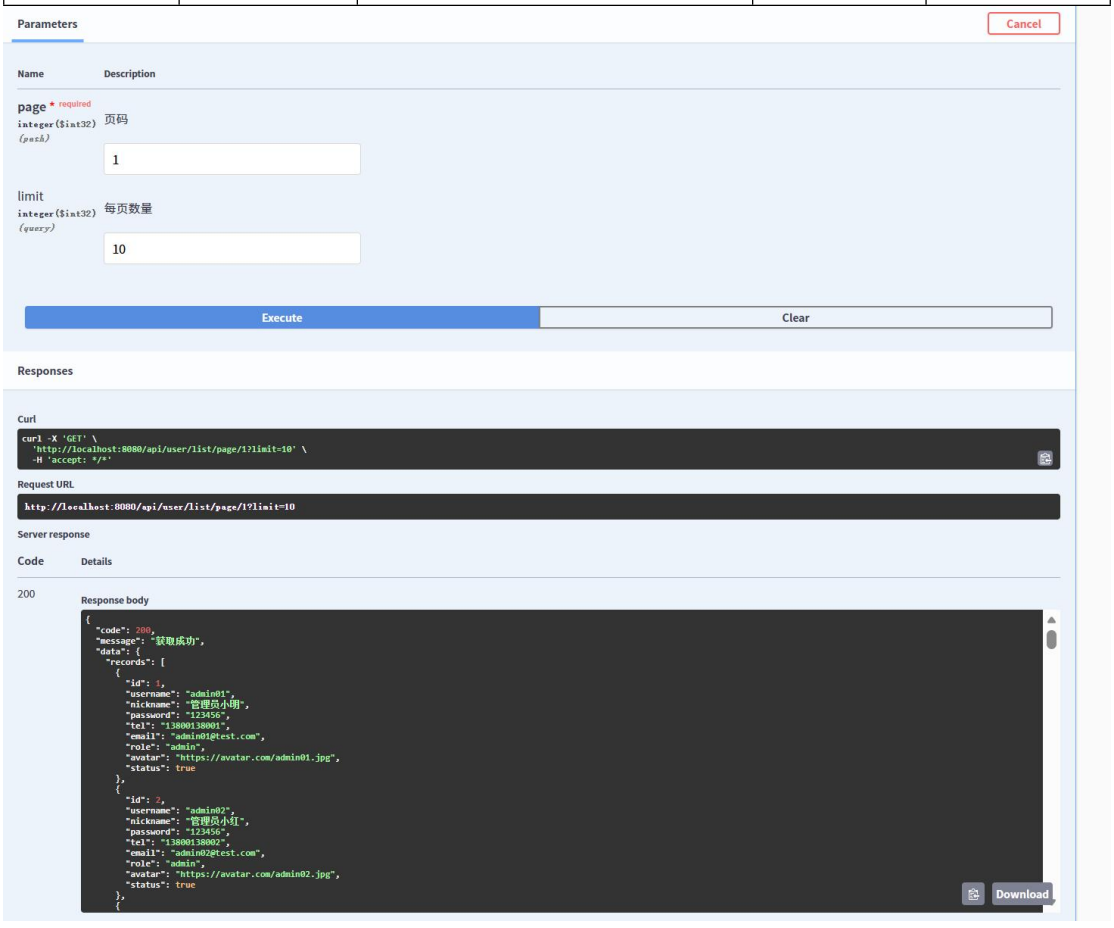
称				

5.11 用户管理

5.11.1 分页获取用户列表

- 测试内容： 验证分页获取用户列表功能
- 测试数据： 页码，每页数量
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
页码,每页数量	分页获取数据	与前端要求不符，分页会返回全部数据	不一致	



5.11.2 获取用户

- 测试内容： 验证根据 id 获取用户功能
- 测试数据： 用户 id
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
用户 id	获取用户	获取用户	一致	

5.11.3 检查用户名是否存在

- 测试内容： 验证注册时验证用户名功能
- 测试数据： 用户名
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
用户名	验证用户名	验证用户名	一致	

5.11.4 获取所有用户

- 测试内容： 验证获取所有用户功能
- 测试数据： 无
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
	获取所有用户	获取所有用户	一致	

5.11.5 用户注册

- 测试内容： 验证新用户注册接口功能
- 测试数据： 用户名，密码，微信昵称，电话，角色身份，用户头像
- 测试方法及步骤： 在 Swagger 输入对应参数，查看输出结果。
- 测试结果：以上各数据的编辑功能均运行正确，满足功能要求。

输入	预期输出	测试输出	结果分析	原因分析
用户名,密码,微信昵称，电话，角色身份，用户头像	注册成功	注册成功	一致	